LAB 1| 22/01/26

Implementation of gates using MCpitts neuron

```
import numpy as np

def mc_pitts_neuron(x, w, theta):
    return 1 if np.dot(x, w) >= theta else 0

# AND gate
w_and = np.array([1, 1])
theta_and = 2

for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:
    print(x, mc_pitts_neuron(np.array(x), w_and, theta_and))

print("Prem Massand,431")
print("Date:22/01/2026")
```

```
(0, 0) 0
(0, 1) 0
(1, 0) 0
(1, 1) 1
Prem Massand,431
Date:22/01/2026
```

```
import numpy as np

def mc_pitts_neuron(x, w, theta):
    return 1 if np.dot(x, w) >= theta else 0

# OR gate
w_or = np.array([1, 1])
theta_or = 1

for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:
    print(x, mc_pitts_neuron(np.array(x), w_or, theta_or))
print("Prem Massand,431")
print("Date:22/01/2026")
```

```
(0, 0) 0
(0, 1) 1
(1, 0) 1
(1, 1) 1
Prem Massand,431
Date:22/01/2026
```

```
import numpy as np
```

```
def mc_pitts_neuron(x, w, theta):
    return 1 if np.dot(x, w) >= theta else 0

# NAND gate
w_nand = np.array([-1, -1])
theta_nand = -1

for x in [(0, 0), (0, 1), (1, 0), (1, 1)]:
    print(x, mc_pitts_neuron(np.array(x), w_nand, theta_nand))
print("Prem Massand,431")
print("Date:22/01/2026")
```

```
(0, 0) 1
(0, 1) 1
(1, 0) 1
(1, 1) 0
Prem Massand,431
Date:22/01/2026
```

Implementation of artificial neuron

```
import numpy as np

def art_neuron(x,w,b,activation='step'):
  z=np.dot(x,w)+b
  if activation=='step':
    return 1 if z>=0 else 0
  elif activation=='sigmoid':
    return 1/(1+np.exp(-z))
  elif activation=='relu':
    return max(0,z)

x=np.array([1,2,3,4])
w=np.array([0.1,0.2,0.3,0.4])
b=0.5
print('step output:',art_neuron(x,w,b,'step'))
print('sigmoid output:',art_neuron(x,w,b,'sigmoid'))
print('relu output:',art_neuron(x,w,b,'relu'))
print("Prem Massand,431")
print("Date:22/01/2026")
```

```
step output: 1
sigmoid output: 0.9706877692486436
relu output: 3.5
Prem Massand,431
Date:22/01/2026
```

```
import numpy as np

def mc_pitts_neuron(x, w, theta):
```

```
    return 1 if np.dot(x, w) >= theta else 0

w_and = np.array([1, 1, 1])
theta_and = 3

for x in [(0, 0,0), (0, 0,1), (0,1,0), (0,1,1),(1,0,0),(1,0,1),
(1,1,0),(1,1,1)]:
    print(x, mc_pitts_neuron(np.array(x), w_and, theta_and))

print("Prem Massand,431")
print("Date:22/01/2026")
```

```
(0, 0, 0) 0
(0, 0, 1) 0
(0, 1, 0) 0
(0, 1, 1) 0
(1, 0, 0) 0
(1, 0, 1) 0
(1, 1, 0) 0
(1, 1, 1) 1
Prem Massand,431
Date:22/01/2026
```

```
import numpy as np

def mc_pitts_neuron(x, w, theta):
    return 1 if np.dot(x, w) >= theta else 0

# NOT gate
w_not = np.array([-1])
theta_not = 0

for x in [(0,), (1,)]:
    print(x, mc_pitts_neuron(np.array(x), w_not, theta_not))

print("Prem Massand,431")
print("Date:22/01/2026")
```

```
(0,) 1
(1,) 0
Prem Massand,431
Date:22/01/2026
```

LAB 2| 29/01/26

```
import numpy as np
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 4
```

```
w,b = np.zeros(2),0

for _ in range(epochs):
  for x,y in zip(x_list,y_list):
    z = np.dot(x,w) + b
    ### condition area for y_pred
    y_pred = 1 if z >= 0 else 0
    E = y - y_pred
    print("error : ",E,"<- at epoch",_)
    w += E*x
    b += E
    print(w,b)


print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  -1 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  1 <- at epoch 0
[1. 1.] 0
error :  -1 <- at epoch 1
[1. 1.] -1
error :  -1 <- at epoch 1
[1. 0.] -2
error :  0 <- at epoch 1
[1. 0.] -2
error :  1 <- at epoch 1
[2. 1.] -1
error :  0 <- at epoch 2
[2. 1.] -1
error :  -1 <- at epoch 2
[2. 0.] -2
error :  -1 <- at epoch 2
[1. 0.] -3
error :  1 <- at epoch 2
[2. 1.] -2
error :  0 <- at epoch 3
[2. 1.] -2
error :  0 <- at epoch 3
[2. 1.] -2
error :  -1 <- at epoch 3
[1. 1.] -3
error :  1 <- at epoch 3
[2. 2.] -2
```

```
Prem Massand,431
Date:29/01/2026
```

```
import numpy as np
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10
w,b = np.zeros(2),0

for _ in range(epochs):
  for x,y in zip(x_list,y_list):
    z = np.dot(x,w) + b
    ### condition area for y_pred
    y_pred = 1 if z >= 0 else 0
    E = y - y_pred
    print("error : ",E,"<- at epoch",_)
    w += E*x
    b += E
    print(w,b)


print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  -1 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  0 <- at epoch 0
[0. 0.] -1
error :  1 <- at epoch 0
[1. 1.] 0
error :  -1 <- at epoch 1
[1. 1.] -1
error :  -1 <- at epoch 1
[1. 0.] -2
error :  0 <- at epoch 1
[1. 0.] -2
error :  1 <- at epoch 1
[2. 1.] -1
error :  0 <- at epoch 2
[2. 1.] -1
error :  -1 <- at epoch 2
[2. 0.] -2
error :  -1 <- at epoch 2
[1. 0.] -3
error :  1 <- at epoch 2
[2. 1.] -2
error :  0 <- at epoch 3
[2. 1.] -2
```

```
error :  0 <- at epoch 3
[2. 1.] -2
error :  -1 <- at epoch 3
[1. 1.] -3
error :  1 <- at epoch 3
[2. 2.] -2
error :  0 <- at epoch 4
[2. 2.] -2
error :  -1 <- at epoch 4
[2. 1.] -3
error :  0 <- at epoch 4
[2. 1.] -3
error :  0 <- at epoch 4
[2. 1.] -3
error :  0 <- at epoch 5
[2. 1.] -3
error :  0 <- at epoch 5
[2. 1.] -3
error :  0 <- at epoch 5
[2. 1.] -3
error :  0 <- at epoch 5
[2. 1.] -3
error :  0 <- at epoch 6
[2. 1.] -3
error :  0 <- at epoch 6
[2. 1.] -3
error :  0 <- at epoch 6
[2. 1.] -3
error :  0 <- at epoch 6
[2. 1.] -3
error :  0 <- at epoch 7
[2. 1.] -3
error :  0 <- at epoch 7
[2. 1.] -3
error :  0 <- at epoch 7
[2. 1.] -3
error :  0 <- at epoch 7
[2. 1.] -3
error :  0 <- at epoch 8
[2. 1.] -3
error :  0 <- at epoch 8
[2. 1.] -3
error :  0 <- at epoch 8
[2. 1.] -3
error :  0 <- at epoch 8
[2. 1.] -3
error :  0 <- at epoch 9
[2. 1.] -3
error :  0 <- at epoch 9
```

```
[2. 1.] -3
error :  0 <- at epoch 9
[2. 1.] -3
error :  0 <- at epoch 9
[2. 1.] -3
Prem Massand,431
Date:29/01/2026
```

```python
import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])

epochs = 10

w = np.array([1, 1])
b = -1

for _ in range(epochs):
    for x, y in zip(x_list, y_list):
        z = np.dot(x, w) + b


        y_pred = 1 if z >= 0 else 0

        E = y - y_pred

        print("error :", E, "<- at epoch", _)

        w += E * x
        b += E

        print(w, b)

print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error : 0 <- at epoch 0
[1 1] -1
error : -1 <- at epoch 0
[1 0] -2
error : 0 <- at epoch 0
[1 0] -2
error : 1 <- at epoch 0
[2 1] -1
error : 0 <- at epoch 1
[2 1] -1
error : -1 <- at epoch 1
[2 0] -2
error : -1 <- at epoch 1
[1 0] -3
```

```
error : 1 <- at epoch 1
[2 1] -2
error : 0 <- at epoch 2
[2 1] -2
error : 0 <- at epoch 2
[2 1] -2
error : -1 <- at epoch 2
[1 1] -3
error : 1 <- at epoch 2
[2 2] -2
error : 0 <- at epoch 3
[2 2] -2
error : -1 <- at epoch 3
[2 1] -3
error : 0 <- at epoch 3
[2 1] -3
error : 0 <- at epoch 3
[2 1] -3
error : 0 <- at epoch 4
[2 1] -3
error : 0 <- at epoch 4
[2 1] -3
error : 0 <- at epoch 4
[2 1] -3
error : 0 <- at epoch 4
[2 1] -3
error : 0 <- at epoch 5
[2 1] -3
error : 0 <- at epoch 5
[2 1] -3
error : 0 <- at epoch 5
[2 1] -3
error : 0 <- at epoch 5
[2 1] -3
error : 0 <- at epoch 6
[2 1] -3
error : 0 <- at epoch 6
[2 1] -3
error : 0 <- at epoch 6
[2 1] -3
error : 0 <- at epoch 6
[2 1] -3
error : 0 <- at epoch 7
[2 1] -3
error : 0 <- at epoch 7
[2 1] -3
error : 0 <- at epoch 7
[2 1] -3
error : 0 <- at epoch 7
```

```
[2 1] -3
error : 0 <- at epoch 8
[2 1] -3
error : 0 <- at epoch 8
[2 1] -3
error : 0 <- at epoch 8
[2 1] -3
error : 0 <- at epoch 8
[2 1] -3
error : 0 <- at epoch 9
[2 1] -3
error : 0 <- at epoch 9
[2 1] -3
error : 0 <- at epoch 9
[2 1] -3
error : 0 <- at epoch 9
[2 1] -3
Prem Massand,431
Date:29/01/2026
```

```python
import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10

w = np.array([1,1])
b = -1

for _ in range(epochs):
    total_error_in_epoch = 0
    for x, y in zip(x_list, y_list):
        z = np.dot(x, w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        total_error_in_epoch += abs(E)
        print("error : ", E, "<- at epoch", _)
        w += E * x
        b += E
        print(w, b)

    if total_error_in_epoch == 0:
        print(f"\nConverged at epoch {_}. No errors found.")
        print("Final Weights:", w)
        print("Final Bias:", b)
        break

print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  0 <- at epoch 0
[1 1] -1
error :  -1 <- at epoch 0
[1 0] -2
error :  0 <- at epoch 0
[1 0] -2
error :  1 <- at epoch 0
[2 1] -1
error :  0 <- at epoch 1
[2 1] -1
error :  -1 <- at epoch 1
[2 0] -2
error :  -1 <- at epoch 1
[1 0] -3
error :  1 <- at epoch 1
[2 1] -2
error :  0 <- at epoch 2
[2 1] -2
error :  0 <- at epoch 2
[2 1] -2
error :  -1 <- at epoch 2
[1 1] -3
error :  1 <- at epoch 2
[2 2] -2
error :  0 <- at epoch 3
[2 2] -2
error :  -1 <- at epoch 3
[2 1] -3
error :  0 <- at epoch 3
[2 1] -3
error :  0 <- at epoch 3
[2 1] -3
error :  0 <- at epoch 4
[2 1] -3
error :  0 <- at epoch 4
[2 1] -3
error :  0 <- at epoch 4
[2 1] -3
error :  0 <- at epoch 4
[2 1] -3

Converged at epoch 4. No errors found.
Final Weights: [2 1]
Final Bias: -3
Prem Massand,431
Date:29/01/2026

import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
```

```python
y_list = np.array([0,1,1,1])

epochs = 10

w = np.array([2,1])
b = -5

for _ in range(epochs):
    total_error_in_epoch = 0
    for x, y in zip(x_list, y_list):
        z = np.dot(x, w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        total_error_in_epoch += abs(E)
        print("error : ", E, "<- at epoch", _)
        w += E * x
        b += E
        print(w, b)

print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  0 <- at epoch 0
[2 1] -5
error :  1 <- at epoch 0
[2 2] -4
error :  1 <- at epoch 0
[3 2] -3
error :  0 <- at epoch 0
[3 2] -3
error :  0 <- at epoch 1
[3 2] -3
error :  1 <- at epoch 1
[3 3] -2
error :  0 <- at epoch 1
[3 3] -2
error :  0 <- at epoch 1
[3 3] -2
error :  0 <- at epoch 2
[3 3] -2
error :  0 <- at epoch 2
[3 3] -2
error :  0 <- at epoch 2
[3 3] -2
error :  0 <- at epoch 2
[3 3] -2
error :  0 <- at epoch 3
[3 3] -2
```

```
error :  0 <- at epoch 3
[3 3] -2
error :  0 <- at epoch 3
[3 3] -2
error :  0 <- at epoch 3
[3 3] -2
error :  0 <- at epoch 4
[3 3] -2
error :  0 <- at epoch 4
[3 3] -2
error :  0 <- at epoch 4
[3 3] -2
error :  0 <- at epoch 4
[3 3] -2
error :  0 <- at epoch 5
[3 3] -2
error :  0 <- at epoch 5
[3 3] -2
error :  0 <- at epoch 5
[3 3] -2
error :  0 <- at epoch 5
[3 3] -2
error :  0 <- at epoch 6
[3 3] -2
error :  0 <- at epoch 6
[3 3] -2
error :  0 <- at epoch 6
[3 3] -2
error :  0 <- at epoch 6
[3 3] -2
error :  0 <- at epoch 7
[3 3] -2
error :  0 <- at epoch 7
[3 3] -2
error :  0 <- at epoch 7
[3 3] -2
error :  0 <- at epoch 7
[3 3] -2
error :  0 <- at epoch 8
[3 3] -2
error :  0 <- at epoch 8
[3 3] -2
error :  0 <- at epoch 8
[3 3] -2
error :  0 <- at epoch 8
[3 3] -2
error :  0 <- at epoch 9
[3 3] -2
error :  0 <- at epoch 9
```

```
[3 3] -2
error :  0 <- at epoch 9
[3 3] -2
error :  0 <- at epoch 9
[3 3] -2
Prem Massand,431
Date:29/01/2026
```

```python
import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])

y_list = np.array([0,1,1,1])

epochs = 10

w = np.array([2,1])
b = -2

for _ in range(epochs):
    total_error_in_epoch = 0
    for x, y in zip(x_list, y_list):
        z = np.dot(x, w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        total_error_in_epoch += abs(E)
        print("error : ", E, "<- at epoch", _)
        w += E * x
        b += E
        print(w, b)

    if total_error_in_epoch == 0:
        print(f"\nConverged at epoch {_}. No errors found.")
        print("Final Weights:", w)
        print("Final Bias:", b)
        break

print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  0 <- at epoch 0
[2 1] -2
error :  1 <- at epoch 0
[2 2] -1
error :  0 <- at epoch 0
[2 2] -1
error :  0 <- at epoch 0
[2 2] -1
error :  0 <- at epoch 1
[2 2] -1
```

```
error :  0 <- at epoch 1
[2 2] -1
error :  0 <- at epoch 1
[2 2] -1
error :  0 <- at epoch 1
[2 2] -1

Converged at epoch 1. No errors found.
Final Weights: [2 2]
Final Bias: -1
Prem Massand,431
Date:29/01/2026
```

```python
import numpy as np

x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,0])

epochs = 10

w = np.array([3,4])
b = -5

for _ in range(epochs):
    total_error_in_epoch = 0
    for x, y in zip(x_list, y_list):
        z = np.dot(x, w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        total_error_in_epoch += abs(E)
        print("error : ", E, "<- at epoch", _)
        w += E * x
        b += E
        print(w, b)

    if total_error_in_epoch == 0:
        print(f"\nConverged at epoch {_}. No errors found.")
        print("Final Weights:", w)
        print("Final Bias:", b)
        break

print("Prem Massand,431")
print("Date:29/01/2026")
```

```
error :  0 <- at epoch 0
[3 4] -5
error :  1 <- at epoch 0
[3 5] -4
error :  1 <- at epoch 0
[4 5] -3
```

```
error :  -1 <- at epoch 0
[3 4] -4
error :  0 <- at epoch 1
[3 4] -4
error :  0 <- at epoch 1
[3 4] -4
error :  1 <- at epoch 1
[4 4] -3
error :  -1 <- at epoch 1
[3 3] -4
error :  0 <- at epoch 2
[3 3] -4
error :  1 <- at epoch 2
[3 4] -3
error :  0 <- at epoch 2
[3 4] -3
error :  -1 <- at epoch 2
[2 3] -4
error :  0 <- at epoch 3
[2 3] -4
error :  1 <- at epoch 3
[2 4] -3
error :  1 <- at epoch 3
[3 4] -2
error :  -1 <- at epoch 3
[2 3] -3
error :  0 <- at epoch 4
[2 3] -3
error :  0 <- at epoch 4
[2 3] -3
error :  1 <- at epoch 4
[3 3] -2
error :  -1 <- at epoch 4
[2 2] -3
error :  0 <- at epoch 5
[2 2] -3
error :  1 <- at epoch 5
[2 3] -2
error :  0 <- at epoch 5
[2 3] -2
error :  -1 <- at epoch 5
[1 2] -3
error :  0 <- at epoch 6
[1 2] -3
error :  1 <- at epoch 6
[1 3] -2
error :  1 <- at epoch 6
[2 3] -1
error :  -1 <- at epoch 6
```

```
[1 2] -2
error :  0 <- at epoch 7
[1 2] -2
error :  0 <- at epoch 7
[1 2] -2
error :  1 <- at epoch 7
[2 2] -1
error :  -1 <- at epoch 7
[1 1] -2
error :  0 <- at epoch 8
[1 1] -2
error :  1 <- at epoch 8
[1 2] -1
error :  0 <- at epoch 8
[1 2] -1
error :  -1 <- at epoch 8
[0 1] -2
error :  0 <- at epoch 9
[0 1] -2
error :  1 <- at epoch 9
[0 2] -1
error :  1 <- at epoch 9
[1 2] 0
error :  -1 <- at epoch 9
[0 1] -1
Prem Massand,431
Date:29/01/2026
```

LAB 3| 05/02/26

```python
import numpy as np

x1, x2 = 0.6, 0.1
y = 1

w1, w2, b = 0.2, -0.3, 0.4
lr = 0.001

def sigmoid(z):
    return 1/(1+np.exp(-z))

z = w1*x1 + w2*x2 + b
y_pred = sigmoid(z)


dL_dy_pred = y_pred - y
```

```
Dy_pred_dz = y_pred*(1-y_pred)

dL_dz = dL_dy_pred * Dy_pred_dz

dL_dw1 = dL_dz * x1
dL_dw2 = dL_dz * x2
dL_db = dL_dz

w1 -= lr*dL_dw1
w2 -= lr*dL_dw2
b  -= lr*dL_db

print(w1, w2, b)


print("Prem Massand,431")
print("Date:05/02/2026")
```

```
0.20005369592979536 -0.29999105067836745 0.4000894932163256
Prem Massand,431
Date:05/02/2026
```

```
import numpy as np

# AND data
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,0,0,1])

w = np.array([0.1, 0.2])
b = 0.3
lr = 0.1

def sigmoid(z):
    return 1/(1+np.exp(-z))


for epoch in range(1000):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = sigmoid(z)


        dz = y_pred - y[i]
        dw = dz * X[i]
        db = dz
```

```
        w -= lr * dw
        b -= lr * db

print("Weights:", w)
print("Bias:", b)


print("Prem Massand,431")
print("Date:05/02/2026")
```

```
Weights: [5.60154076 5.59544869]
Bias: -8.565987677634602
Prem Massand,431
Date:05/02/2026
```

```
import numpy as np

# OR data
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0, 1, 1, 1])

w = np.array([0.1, 0.2])
b = 0.3
lr = 0.1

def sigmoid(z):
    return 1/(1+np.exp(-z))

for epoch in range(1000):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = sigmoid(z)

        dz = y_pred - y[i]
        dw = dz * X[i]
        db = dz

        w -= lr * dw
        b -= lr * db

print("Weights:", w)
print("Bias:", b)
```

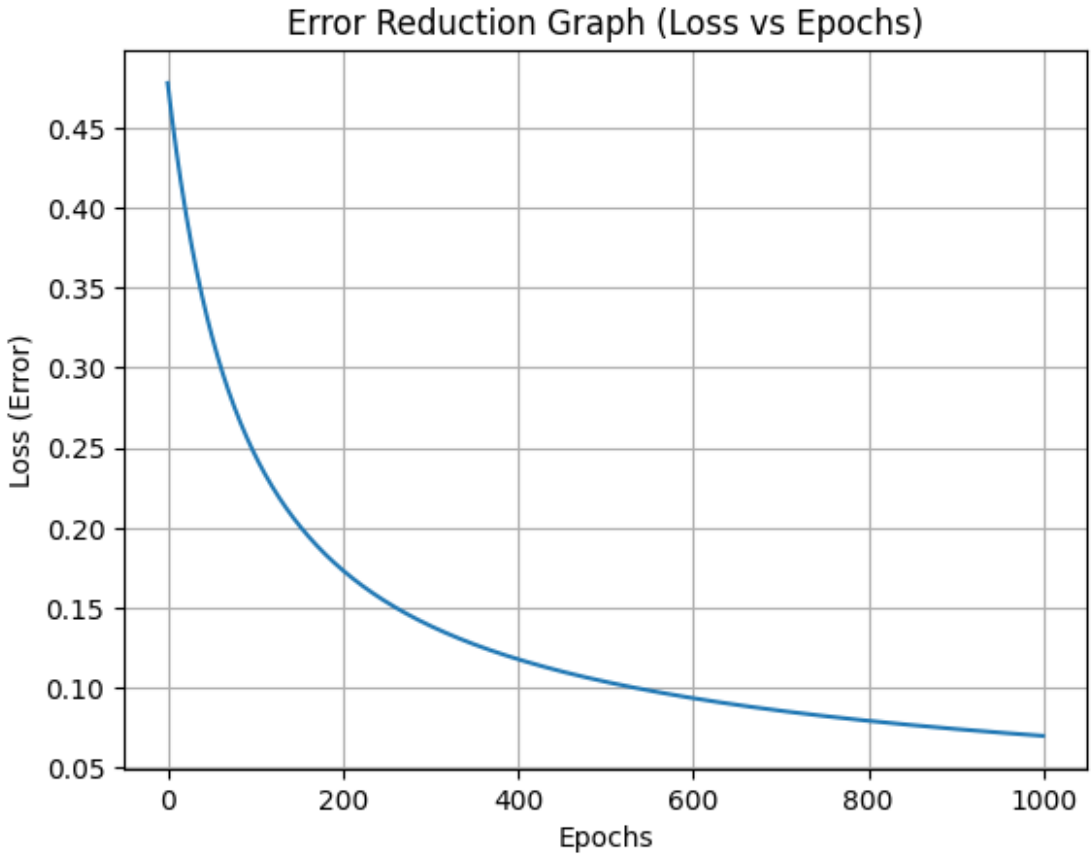
```
print("Prem Massand,431")
print("Date:05/02/2026")
```

```
Weights: [6.7832796  6.78742098]
Bias: -2.9139465005185365
Prem Massand,431
Date:05/02/2026
```

```
import numpy as np
import matplotlib.pyplot as plt

# Inputs
x1, x2 = 0.6, 0.1
y = 1

w1, w2, b = 0.2, -0.3, 0.4
lr = 0.1

def sigmoid(z):
    return 1/(1+np.exp(-z))

loss_list = []

epochs = 1000

for epoch in range(epochs):


    z = w1*x1 + w2*x2 + b
    y_pred = sigmoid(z)


    loss = -(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
    loss_list.append(loss)


    dL_dy_pred = y_pred - y
    Dy_pred_dz = y_pred * (1 - y_pred)
    dL_dz = dL_dy_pred * Dy_pred_dz

    dL_dw1 = dL_dz * x1
    dL_dw2 = dL_dz * x2
    dL_db  = dL_dz

    # Update
    w1 -= lr * dL_dw1
    w2 -= lr * dL_dw2
    b  -= lr * dL_db

print("Final w1:", w1)
print("Final w2:", w2)
```

```
print("Final b :", b)

plt.plot(loss_list)
plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("Error Reduction Graph (Loss vs Epochs)")
plt.grid(True)
plt.show()

print("Prem Massand,431")
print("Date:05/02/2026")
```

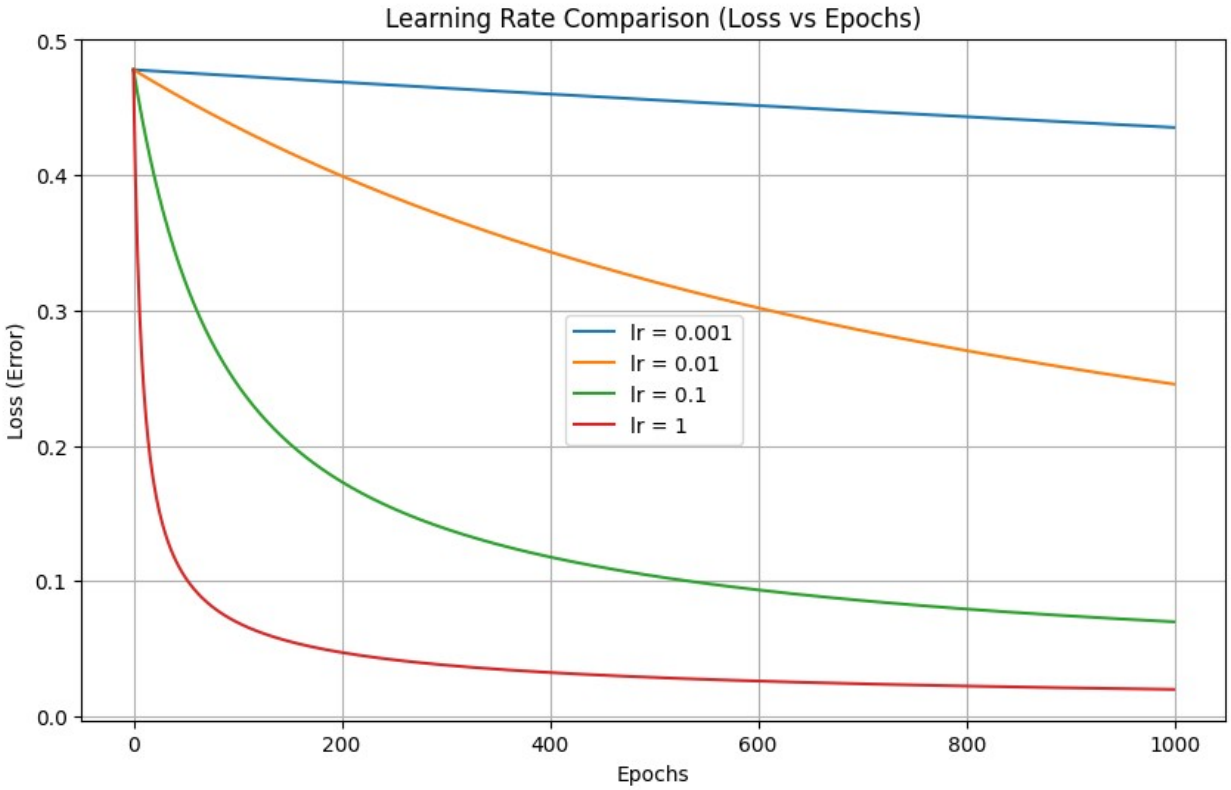
```
Final w1: 1.134656047343927
Final w2: -0.14422399210934486
Final b : 1.9577600789065457
```




```
Prem Massand,431
Date:05/02/2026

import numpy as np
import matplotlib.pyplot as plt

# Inputs
```

```
x1, x2 = 0.6, 0.1
y = 1

def sigmoid(z):
    return 1/(1+np.exp(-z))

learning_rates = [0.001, 0.01, 0.1, 1]

epochs = 1000

plt.figure(figsize=(10, 6))

for lr in learning_rates:


    w1, w2, b = 0.2, -0.3, 0.4

    loss_list = []

    for epoch in range(epochs):


        z = w1*x1 + w2*x2 + b
        y_pred = sigmoid(z)


        loss = -(y*np.log(y_pred) + (1-y)*np.log(1-y_pred))
        loss_list.append(loss)


        dL_dy_pred = y_pred - y
        Dy_pred_dz = y_pred * (1 - y_pred)
        dL_dz = dL_dy_pred * Dy_pred_dz

        dL_dw1 = dL_dz * x1
        dL_dw2 = dL_dz * x2
        dL_db  = dL_dz


        w1 -= lr * dL_dw1
        w2 -= lr * dL_dw2
        b  -= lr * dL_db


    plt.plot(loss_list, label=f"lr = {lr}")

plt.xlabel("Epochs")
plt.ylabel("Loss (Error)")
plt.title("Learning Rate Comparison (Loss vs Epochs)")
plt.grid(True)
plt.legend()
```

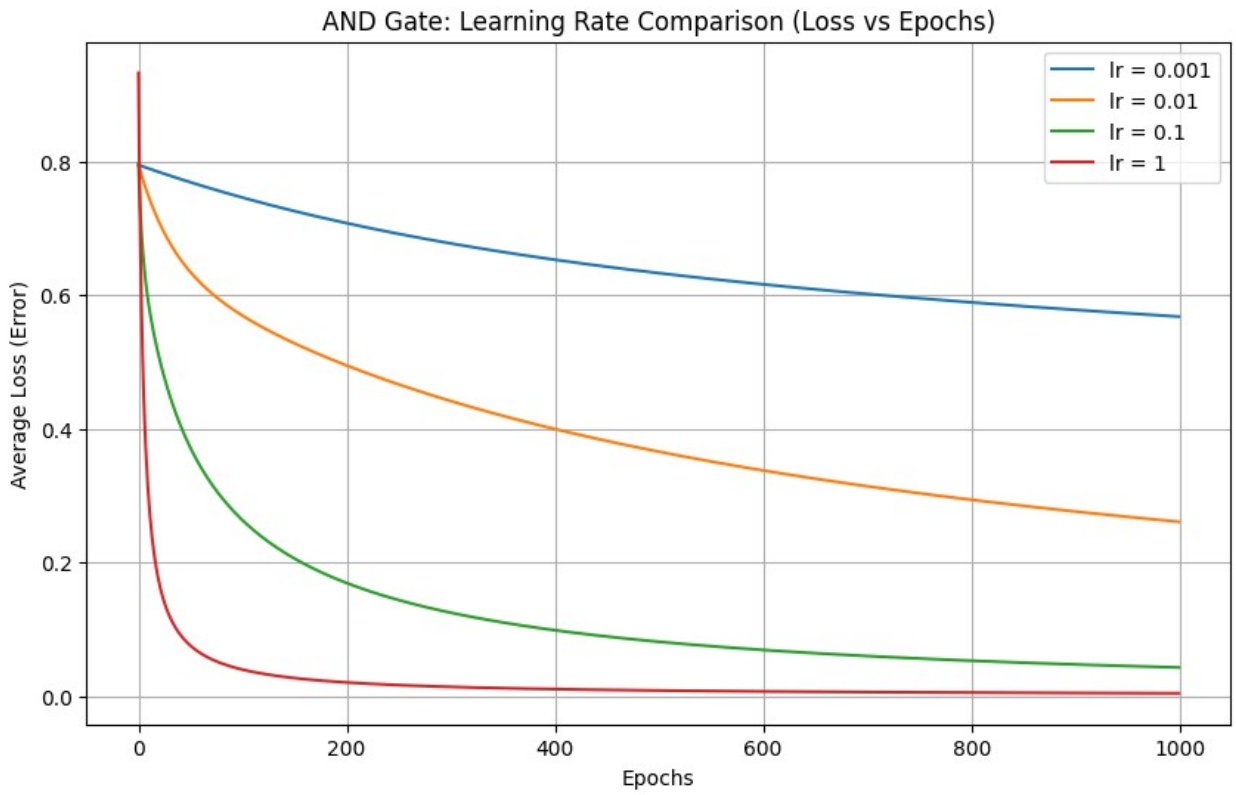
```
plt.show()

print("Prem Massand,431")
print("Date:05/02/2026")
```




```
Prem Massand,431
Date:05/02/2026

import numpy as np
import matplotlib.pyplot as plt

# AND data
X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0,0,0,1])

def sigmoid(z):
    return 1/(1+np.exp(-z))

learning_rates = [0.001, 0.01, 0.1, 1]
```

```
epochs = 1000

plt.figure(figsize=(10, 6))

for lr in learning_rates:


    w = np.array([0.1, 0.2], dtype=float)
    b = 0.3

    loss_list = []

    for epoch in range(epochs):

        total_loss = 0

        for i in range(len(X)):


            z = np.dot(X[i], w) + b
            y_pred = sigmoid(z)


            loss = -(y[i]*np.log(y_pred) + (1-y[i])*np.log(1-y_pred))
            total_loss += loss


            dz = y_pred - y[i]
            dw = dz * X[i]
            db = dz


            w -= lr * dw
            b -= lr * db


        loss_list.append(total_loss / len(X))


    plt.plot(loss_list, label=f"lr = {lr}")

plt.xlabel("Epochs")
plt.ylabel("Average Loss (Error)")
plt.title("AND Gate: Learning Rate Comparison (Loss vs Epochs)")
plt.grid(True)
plt.legend()
plt.show()

lr = 0.1
w = np.array([0.1, 0.2], dtype=float)
```

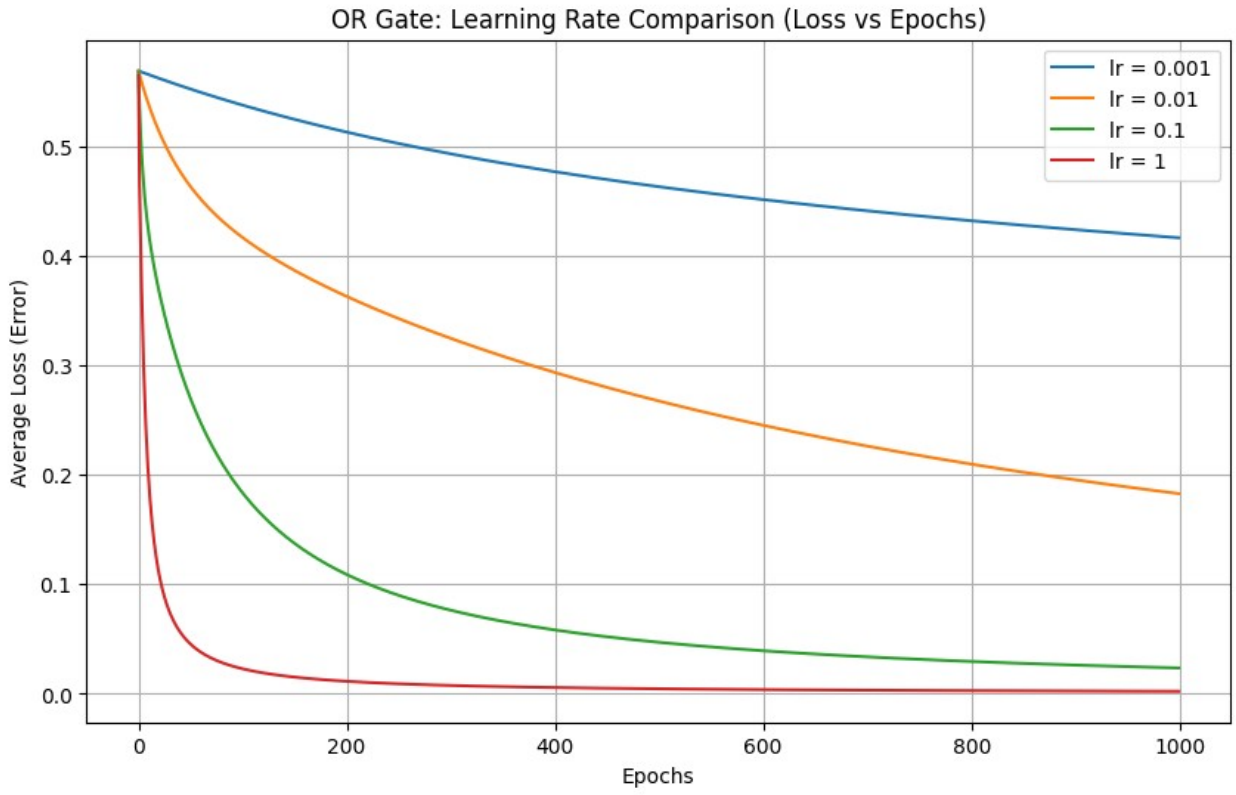
```
b = 0.3

for epoch in range(1000):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = sigmoid(z)

        dz = y_pred - y[i]
        dw = dz * X[i]
        db = dz

        w -= lr * dw
        b -= lr * db

print("Weights:", w)
print("Bias:", b)

print("Prem Massand,431")
print("Date:05/02/2026")
```




```
Weights: [5.60154076 5.59544869]
Bias: -8.565987677634602
Prem Massand,431
Date:05/02/2026
```

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

y = np.array([0, 1, 1, 1])

def sigmoid(z):
    return 1/(1+np.exp(-z))

learning_rates = [0.001, 0.01, 0.1, 1]

epochs = 1000

plt.figure(figsize=(10, 6))

for lr in learning_rates:


    w = np.array([0.1, 0.2], dtype=float)
    b = 0.3

    loss_list = []

    for epoch in range(epochs):

        total_loss = 0

        for i in range(len(X)):


            z = np.dot(X[i], w) + b
            y_pred = sigmoid(z)


            loss = -(y[i]*np.log(y_pred) + (1-y[i])*np.log(1-y_pred))
            total_loss += loss


            dz = y_pred - y[i]
            dw = dz * X[i]
            db = dz


            w -= lr * dw
            b -= lr * db
```

```
        loss_list.append(total_loss / len(X))

    plt.plot(loss_list, label=f"lr = {lr}")

plt.xlabel("Epochs")
plt.ylabel("Average Loss (Error)")
plt.title("OR Gate: Learning Rate Comparison (Loss vs Epochs)")
plt.grid(True)
plt.legend()
plt.show()

lr = 0.1
w = np.array([0.1, 0.2], dtype=float)
b = 0.3

for epoch in range(1000):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = sigmoid(z)

        dz = y_pred - y[i]
        dw = dz * X[i]
        db = dz

        w -= lr * dw
        b -= lr * db

print("Weights:", w)
print("Bias:", b)

print("Prem Massand,431")
print("Date:05/02/2026")
```




```
Weights: [6.7832796  6.78742098]
Bias: -2.9139465005185365
Prem Massand,431
Date:05/02/2026
```

LAB 4| 12/02/2026

```
import pandas as pd

df = pd.read_csv("spam_or_not_spam.csv")

print("Shape:", df.shape)
print(df.head())
print(df["label"].value_counts())

Shape: (3000, 2)
                                               email  label
0   date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...      0
1  martin a posted tassos papadopoulos the greek ...      0
2  man threatens explosion in moscow thursday aug...      0
3  klez the virus that won t die already the most...      0
4   in adding cream to spaghetti carbonara which ...      0
label
0    2500
```

```
1     500
Name: count, dtype: int64
```

```
from sklearn.model_selection import train_test_split

df = df.dropna(subset=["email"])

X = df["email"]
y = df["label"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train size:", len(X_train))
print("Test size:", len(X_test))
```

```
Train size: 2399
Test size: 600
```

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words="english", max_features=5000)

X_train_vec = vectorizer.fit_transform(X_train).toarray()
X_test_vec = vectorizer.transform(X_test).toarray()

print("Train vector shape:", X_train_vec.shape)
print("Test vector shape:", X_test_vec.shape)
```

```
Train vector shape: (2399, 5000)
Test vector shape: (600, 5000)
```

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y_true, y_pred):
    eps = 1e-9
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1
- y_pred))

def train_single_neuron(X_train, y_train, X_test, y_test, lr=0.1,
epochs=50):
```

```
    n_features = X_train.shape[1]
    w = np.zeros(n_features)
    b = 0

    losses = []
    train_accs = []

    for epoch in range(epochs):


        z = np.dot(X_train, w) + b
        y_hat = sigmoid(z)


        loss = binary_cross_entropy(y_train, y_hat)


        y_pred_train = (y_hat >= 0.5).astype(int)
        train_acc = np.mean(y_pred_train == y_train)


        dw = np.dot(X_train.T, (y_hat - y_train)) / len(y_train)
        db = np.mean(y_hat - y_train)


        w = w - lr * dw
        b = b - lr * db

        losses.append(loss)
        train_accs.append(train_acc)


    z_test = np.dot(X_test, w) + b
    y_hat_test = sigmoid(z_test)
    y_pred_test = (y_hat_test >= 0.5).astype(int)
    test_acc = np.mean(y_pred_test == y_test)

    return losses, train_accs, test_acc

z_test = np.dot(X_test_vec, w) + b
y_hat_test = sigmoid(z_test)

y_pred_test = (y_hat_test >= 0.5).astype(int)

test_acc = np.mean(y_pred_test == y_test)
print("Test Accuracy:", test_acc)
```

```
Test Accuracy: 0.8333333333333334
```

```
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(y_test, y_pred_test)
```

```
recall = recall_score(y_test, y_pred_test)
f1 = f1_score(y_test, y_pred_test)

print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

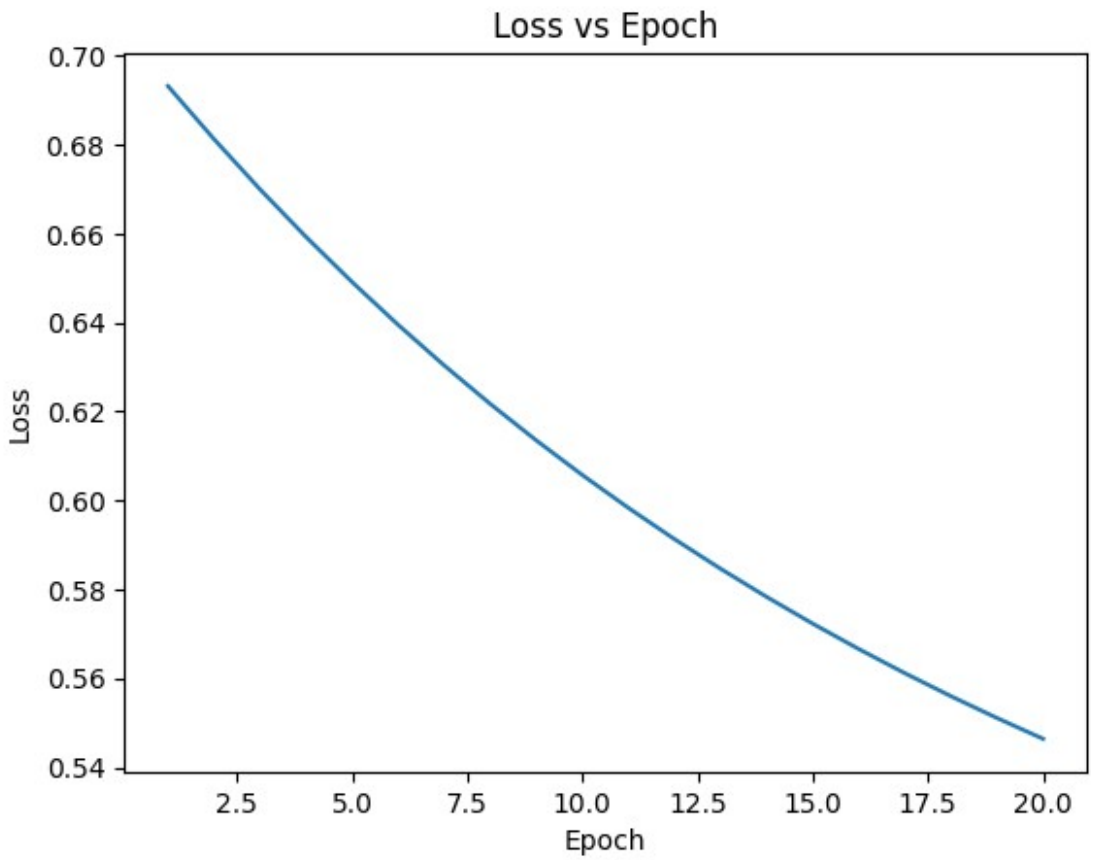
```
Precision: 0.0
Recall: 0.0
F1 Score: 0.0
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/
_classification.py:1565: UndefinedMetricWarning: Precision is ill-
defined and being set to 0.0 due to no predicted samples. Use
`zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is",
len(result))
```

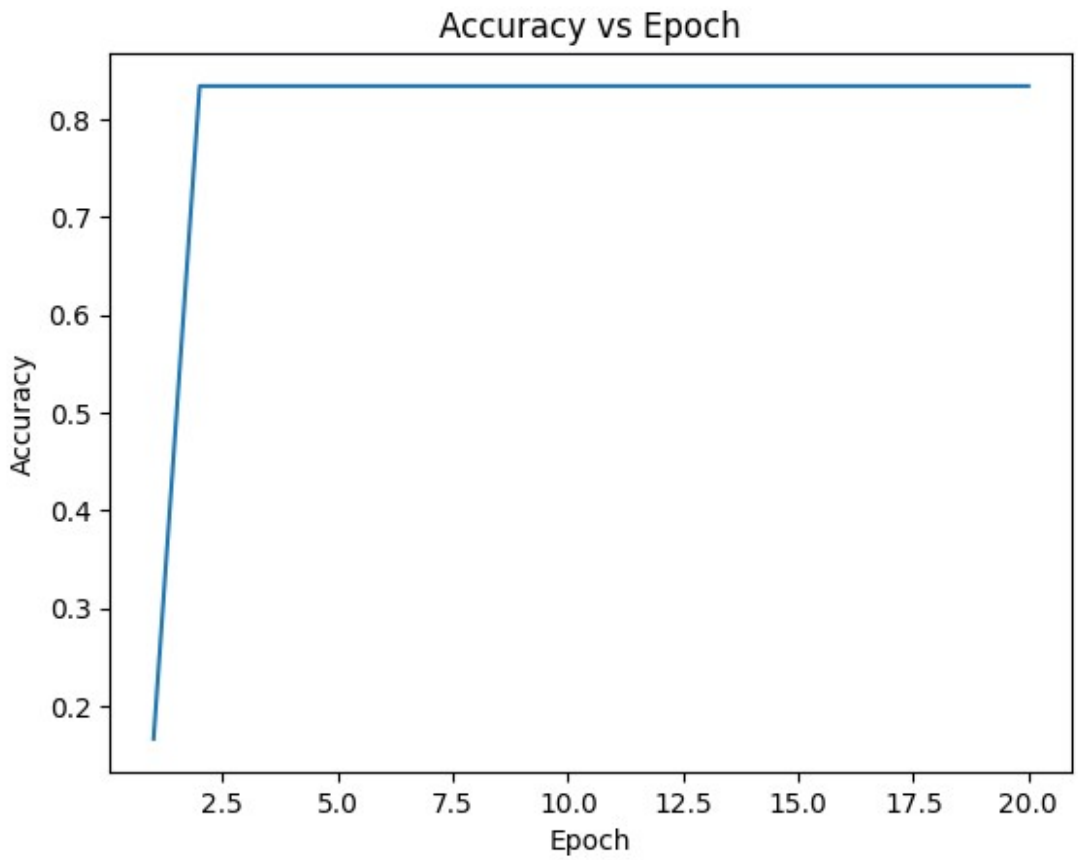
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred_test)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[500   0]
 [100   0]]
```

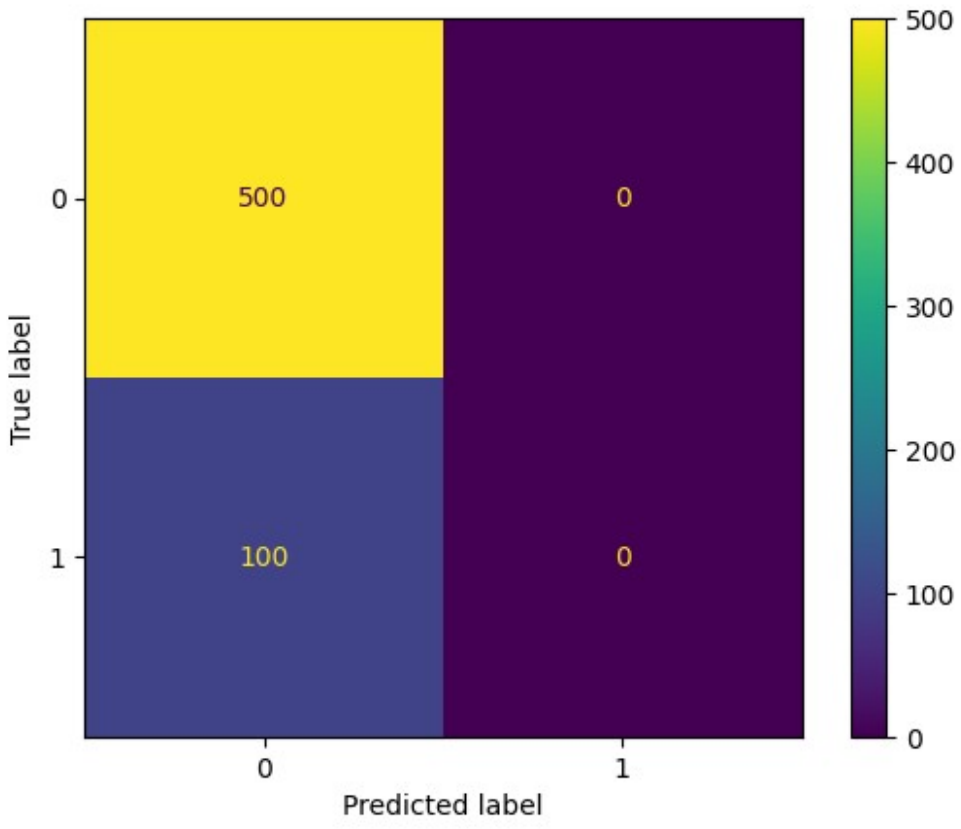
```
import matplotlib.pyplot as plt

plt.plot(range(1, epochs+1), losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.show()
```



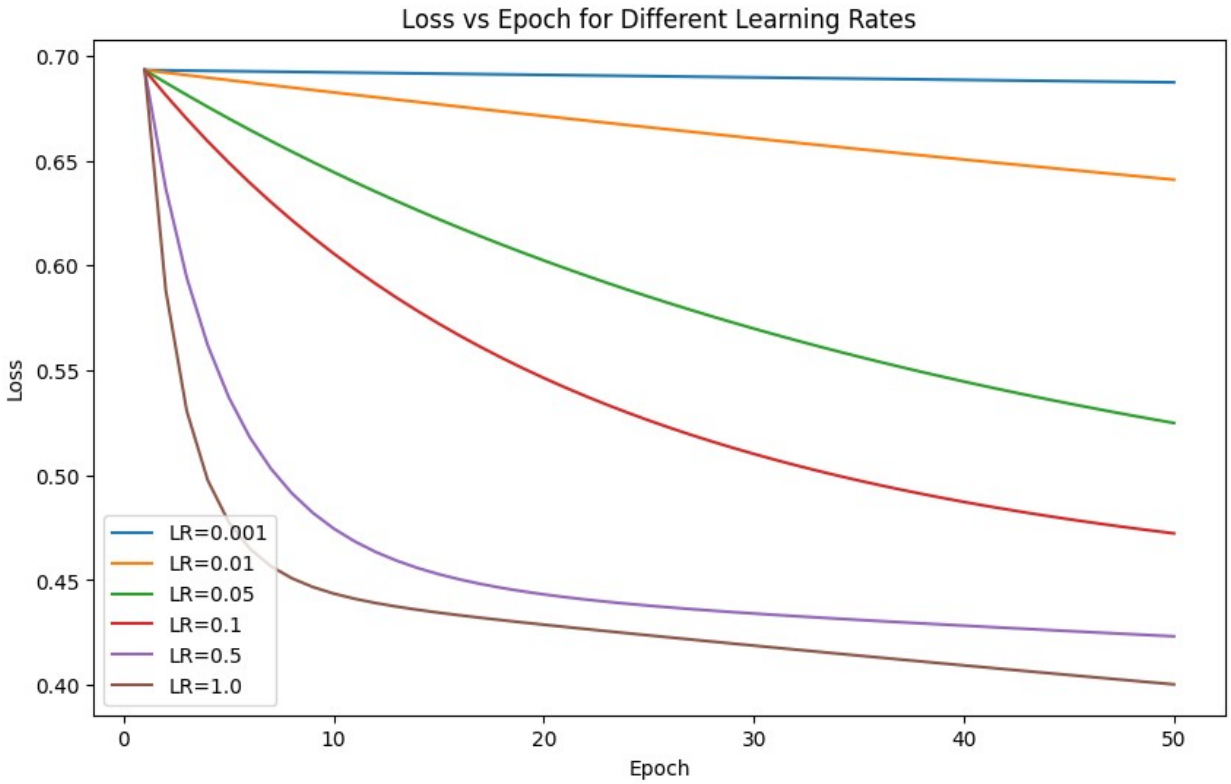

```
plt.plot(range(1, epochs+1), accs)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Epoch")
plt.show()
```



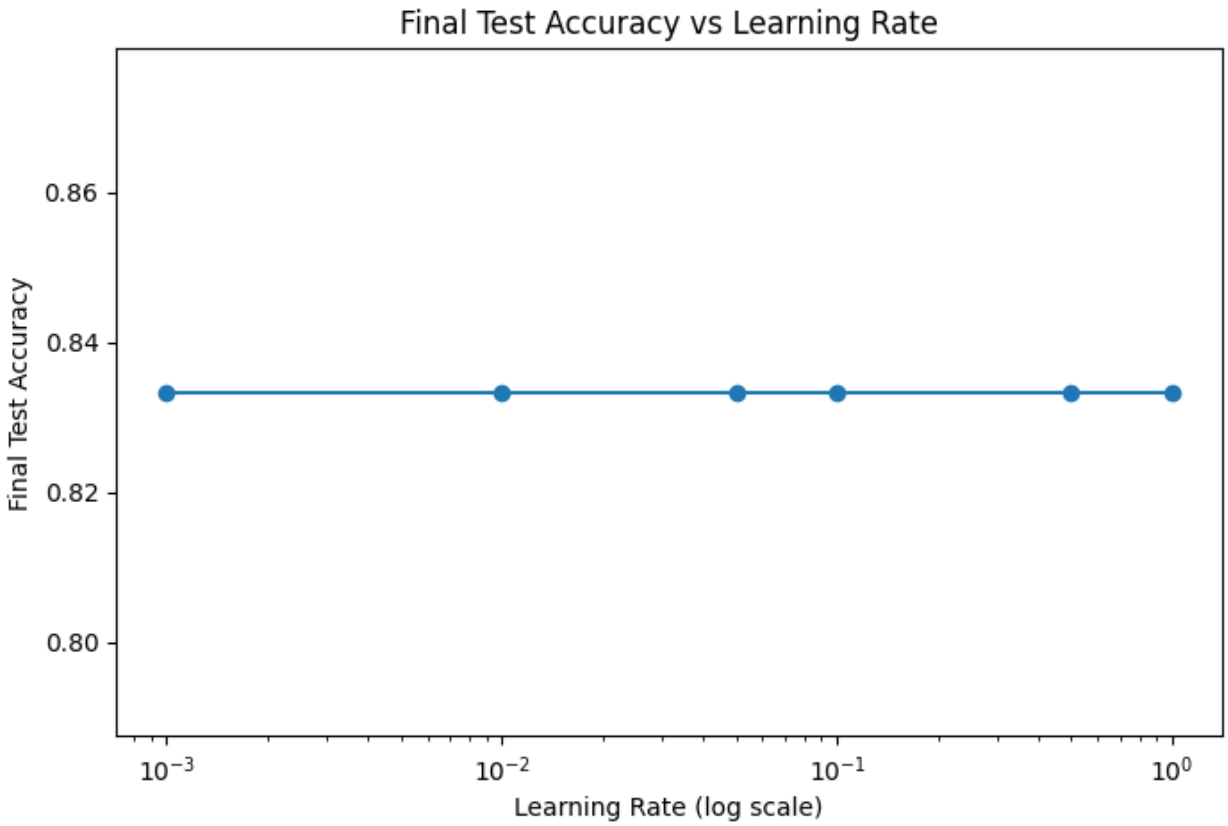

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.show()
```




```
learning_rates = [0.001, 0.01, 0.05, 0.1, 0.5, 1.0]
epochs = 50

final_test_acc = {}

plt.figure(figsize=(10,6))

for lr in learning_rates:
    losses, train_accs, test_acc = train_single_neuron(
        X_train_vec, y_train, X_test_vec, y_test,
        lr=lr, epochs=epochs
    )

    final_test_acc[lr] = test_acc
    plt.plot(range(1, epochs+1), losses, label=f"LR={lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```




```
lrs = list(final_test_acc.keys())
accs = list(final_test_acc.values())

plt.figure(figsize=(8,5))
plt.plot(lrs, accs, marker="o")

plt.xscale("log")  # IMPORTANT (because LR values are exponential)
plt.xlabel("Learning Rate (log scale)")
plt.ylabel("Final Test Accuracy")
plt.title("Final Test Accuracy vs Learning Rate")
plt.show()

print("Final Test Accuracy for each LR:")
for lr, acc in final_test_acc.items():
    print(f"LR={lr}  ->  Test Accuracy={acc:.4f}")
```




```
Final Test Accuracy for each LR:
LR=0.001  ->  Test Accuracy=0.8333
LR=0.01  ->  Test Accuracy=0.8333
LR=0.05  ->  Test Accuracy=0.8333
LR=0.1  ->  Test Accuracy=0.8333
LR=0.5  ->  Test Accuracy=0.8333
LR=1.0  ->  Test Accuracy=0.8333
```

LAB 5

```python
import numpy as np

# XOR dataset (4 input combinations)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

# XOR output
y = np.array([
```

```
    [0],
    [1],
    [1],
    [0]
])

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Set random seed for same results
np.random.seed(42)

# Network architecture
input_neurons = 2
hidden_neurons = 2
output_neurons = 1

# Initialize weights and biases
W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.random.randn(1, hidden_neurons)

W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.random.randn(1, output_neurons)

# Hyperparameters
learning_rate = 0.1
epochs = 10000

# Training loop
for epoch in range(epochs):

    # ---------- Forward Propagation ----------
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    loss = np.mean((y - final_output) ** 2)

    # ---------- Backpropagation ----------
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W2.T)
```

```
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # ---------- Update Weights and Biases ----------
    W2 += hidden_output.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate

    W1 += X.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.6f}")

print("\nFinal Predictions:")
for i in range(len(X)):
    prediction = final_output[i][0]
    predicted_class = 1 if prediction >= 0.5 else 0
    print(f"Input: {X[i]} -> Predicted: {prediction:.4f} -> Class:
{predicted_class}")
```

```
Epoch 0, Loss: 0.294351
Epoch 1000, Loss: 0.244410
Epoch 2000, Loss: 0.203539
Epoch 3000, Loss: 0.153349
Epoch 4000, Loss: 0.046331
Epoch 5000, Loss: 0.015615
Epoch 6000, Loss: 0.008448
Epoch 7000, Loss: 0.005614
Epoch 8000, Loss: 0.004146
Epoch 9000, Loss: 0.003264

Final Predictions:
Input: [0 0] -> Predicted: 0.0540 -> Class: 0
Input: [0 1] -> Predicted: 0.9505 -> Class: 1
Input: [1 0] -> Predicted: 0.9501 -> Class: 1
Input: [1 1] -> Predicted: 0.0536 -> Class: 0
```

LAB 8| 12 March 2026

Change the batch size

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Load dataset
```

```python
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Different batch sizes
batch_sizes = [32, 64, 128, 256]

for batch in batch_sizes:
    print("\nTraining with batch size:", batch)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
    ])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    model.summary()

    # Train model
    model.fit(X_train, y_train, epochs=15, batch_size=batch)

    # Evaluate
    test_loss, test_acc = model.evaluate(X_test, y_test)
    print("Test accuracy with batch size", batch, ":", test_acc)
```

```
Training with batch size: 32

Model: "sequential_12"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━━┩
│ dense_12 (Dense)                     │ (None, 10)                  │
7,850 │
```

```
└─────────────────────────────────────┴─────────────────────────────┴──────────
──────┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.8088 - loss:
0.7304
Epoch 2/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9118 - loss:
0.3107
Epoch 3/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9201 - loss:
0.2881
Epoch 4/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9231 - loss:
0.2704
Epoch 5/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9262 - loss:
0.2650
Epoch 6/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9287 - loss:
0.2600
Epoch 7/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9285 - loss:
0.2563
Epoch 8/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9315 - loss:
0.2504
Epoch 9/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9302 - loss:
0.2547
Epoch 10/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9306 - loss:
0.2523
Epoch 11/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9324 - loss:
0.2391
Epoch 12/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9327 - loss:
0.2431
Epoch 13/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9320 - loss:
0.2450
Epoch 14/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9329 - loss:
```

```
0.2422
Epoch 15/15
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9322 - loss:
0.2469
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9167 - loss:
0.3005
Test accuracy with batch size 32 : 0.9271000027656555

Training with batch size: 64

Model: "sequential_13"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━━┩
│ dense_13 (Dense)                     │ (None, 10)                  │
7,850 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
━━━━┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7833 - loss:
0.8541
Epoch 2/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9094 - loss:
0.3327
Epoch 3/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9193 - loss:
0.2937
Epoch 4/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9206 - loss:
0.2841
Epoch 5/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9221 - loss:
0.2753
Epoch 6/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9267 - loss:
0.2660
Epoch 7/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9266 - loss:
0.2623
Epoch 8/15
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9277 - loss:
0.2570
Epoch 9/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9300 - loss:
0.2561
Epoch 10/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9313 - loss:
0.2456
Epoch 11/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9315 - loss:
0.2451
Epoch 12/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9313 - loss:
0.2512
Epoch 13/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9309 - loss:
0.2535
Epoch 14/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9323 - loss:
0.2493
Epoch 15/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9329 - loss:
0.2413
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9166 - loss:
0.2993
Test accuracy with batch size 64 : 0.927299976348877

Training with batch size: 128

Model: "sequential_14"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━┩
│ dense_14 (Dense)                     │ (None, 10)                  │
7,850 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
──┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.7391 - loss:
1.0215
```

```
Epoch 2/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8985 - loss:
0.3783
Epoch 3/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9120 - loss:
0.3196
Epoch 4/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9164 - loss:
0.2994
Epoch 5/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9208 - loss:
0.2842
Epoch 6/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9225 - loss:
0.2793
Epoch 7/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9235 - loss:
0.2753
Epoch 8/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9255 - loss:
0.2661
Epoch 9/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9274 - loss:
0.2680
Epoch 10/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9277 - loss:
0.2588
Epoch 11/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9287 - loss:
0.2613
Epoch 12/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9292 - loss:
0.2557
Epoch 13/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9291 - loss:
0.2520
Epoch 14/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9296 - loss:
0.2493
Epoch 15/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9332 - loss:
0.2457
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9160 - loss:
0.2997
Test accuracy with batch size 128 : 0.9275000095367432

Training with batch size: 256

Model: "sequential_15"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ dense_15 (Dense)                │ (None, 10)             │         7,850 │
└─────────────────────────────────┴────────────────────────┴───────────────┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.6377 - loss:
1.2971
Epoch 2/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8837 - loss:
0.4535
Epoch 3/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8992 - loss:
0.3755
Epoch 4/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9098 - loss:
0.3316
Epoch 5/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9129 - loss:
0.3177
Epoch 6/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9171 - loss:
0.3057
Epoch 7/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9196 - loss:
0.2886
Epoch 8/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9242 - loss:
0.2769
Epoch 9/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9214 - loss:
0.2820
Epoch 10/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9228 - loss:
0.2760
Epoch 11/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9251 - loss:
0.2666
```

```
Epoch 12/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9243 - loss:
0.2694
Epoch 13/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9269 - loss:
0.2626
Epoch 14/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9295 - loss:
0.2591
Epoch 15/15
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9277 - loss:
0.2611
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9154 - loss:
0.3028
Test accuracy with batch size 256 : 0.9259999990463257
```

Change the number of epochs

```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Different epoch values
epochs_list = [10, 15, 20]

for ep in epochs_list:

    print("\nTraining with epochs:", ep)

    # Model
    model = Sequential([
        Dense(10, activation='softmax', input_shape=(784,))
```

```
    ])

    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    model.summary()

    # Train model
    model.fit(X_train, y_train, epochs=ep, batch_size=64)

    # Evaluate
    test_loss, test_acc = model.evaluate(X_test, y_test)

    print("Epochs:", ep, "| Test accuracy:", test_acc)


Training with epochs: 10

Model: "sequential_30"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ dense_30 (Dense)                     │ (None, 10)                  │
7,850 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.7850 - loss:
0.8524
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9090 - loss:
0.3332
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9157 - loss:
0.3022
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9216 - loss:
0.2814
Epoch 5/10
```

```
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9236 - loss:
0.2753
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9269 - loss:
0.2673
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9271 - loss:
0.2605
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9283 - loss:
0.2587
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9290 - loss:
0.2551
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9298 - loss:
0.2538
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9147 - loss:
0.3030
Epochs: 10 | Test accuracy: 0.9258999824523926

Training with epochs: 15

Model: "sequential_31"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━━━┩
│ dense_31 (Dense)                     │ (None, 10)                  │
7,850 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
────┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7792 - loss:
0.8587
Epoch 2/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9078 - loss:
0.3371
Epoch 3/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9151 - loss:
0.3043
```

```
Epoch 4/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9201 - loss:
0.2829
Epoch 5/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9223 - loss:
0.2750
Epoch 6/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9257 - loss:
0.2676
Epoch 7/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9270 - loss:
0.2629
Epoch 8/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9280 - loss:
0.2606
Epoch 9/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9284 - loss:
0.2577
Epoch 10/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9309 - loss:
0.2523
Epoch 11/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9302 - loss:
0.2521
Epoch 12/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9311 - loss:
0.2464
Epoch 13/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9309 - loss:
0.2457
Epoch 14/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9339 - loss:
0.2457
Epoch 15/15
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9333 - loss:
0.2439
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9169 - loss:
0.2978
Epochs: 15 | Test accuracy: 0.9269999861717224

Training with epochs: 20

Model: "sequential_32"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
━━┩
```

```
| dense_32 (Dense)                 | (None, 10)              |
7,850 |
└──────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.7607 - loss:
0.8978
Epoch 2/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9070 - loss:
0.3359
Epoch 3/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9167 - loss:
0.3020
Epoch 4/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9226 - loss:
0.2795
Epoch 5/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9236 - loss:
0.2766
Epoch 6/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 2ms/step - accuracy: 0.9261 - loss:
0.2666
Epoch 7/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9261 - loss:
0.2650
Epoch 8/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9275 - loss:
0.2610
Epoch 9/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9284 - loss:
0.2624
Epoch 10/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9293 - loss:
0.2523
Epoch 11/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9301 - loss:
0.2544
Epoch 12/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - accuracy: 0.9314 - loss:
0.2493
Epoch 13/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9333 - loss:
0.2445
```

```
Epoch 14/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9294 - loss:
0.2522
Epoch 15/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9329 - loss:
0.2411
Epoch 16/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9322 - loss:
0.2438
Epoch 17/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9346 - loss:
0.2368
Epoch 18/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9318 - loss:
0.2456
Epoch 19/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9340 - loss:
0.2398
Epoch 20/20
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9359 - loss:
0.2352
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9177 - loss:
0.2992
Epochs: 20 | Test accuracy: 0.9283000230789185
```

Accuracy and Loss curves

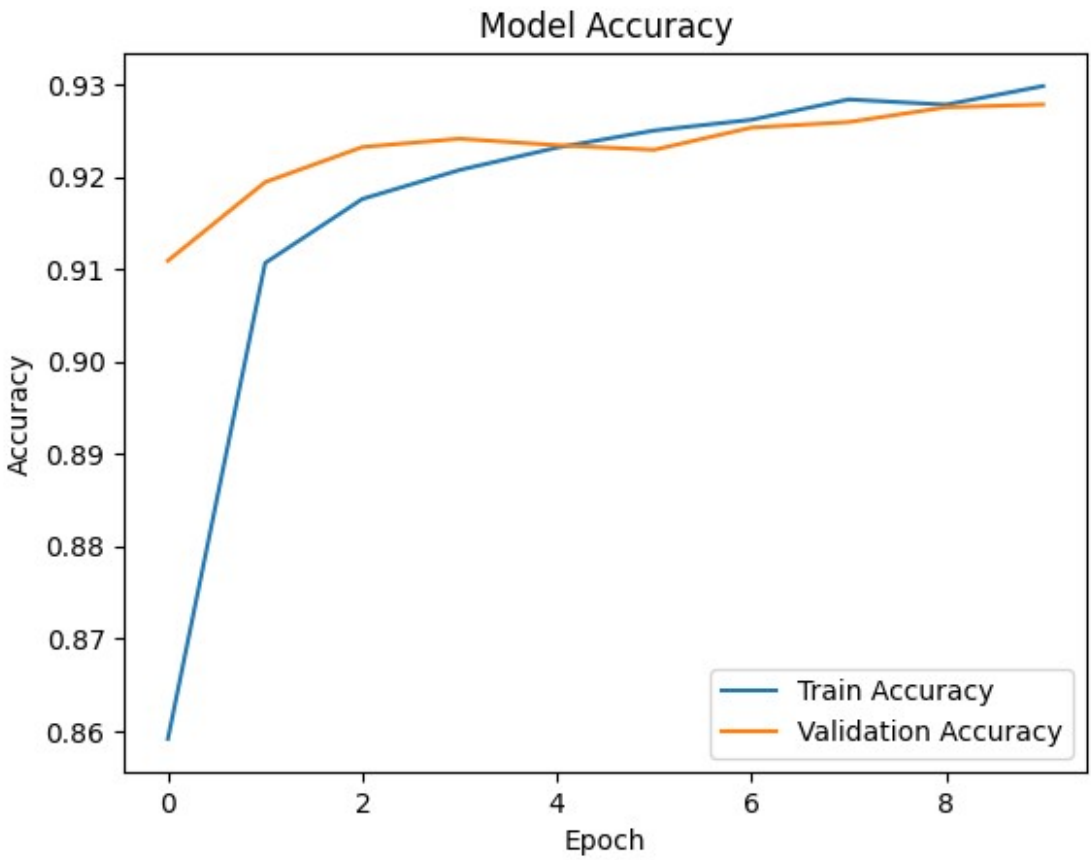
```python
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Flatten
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

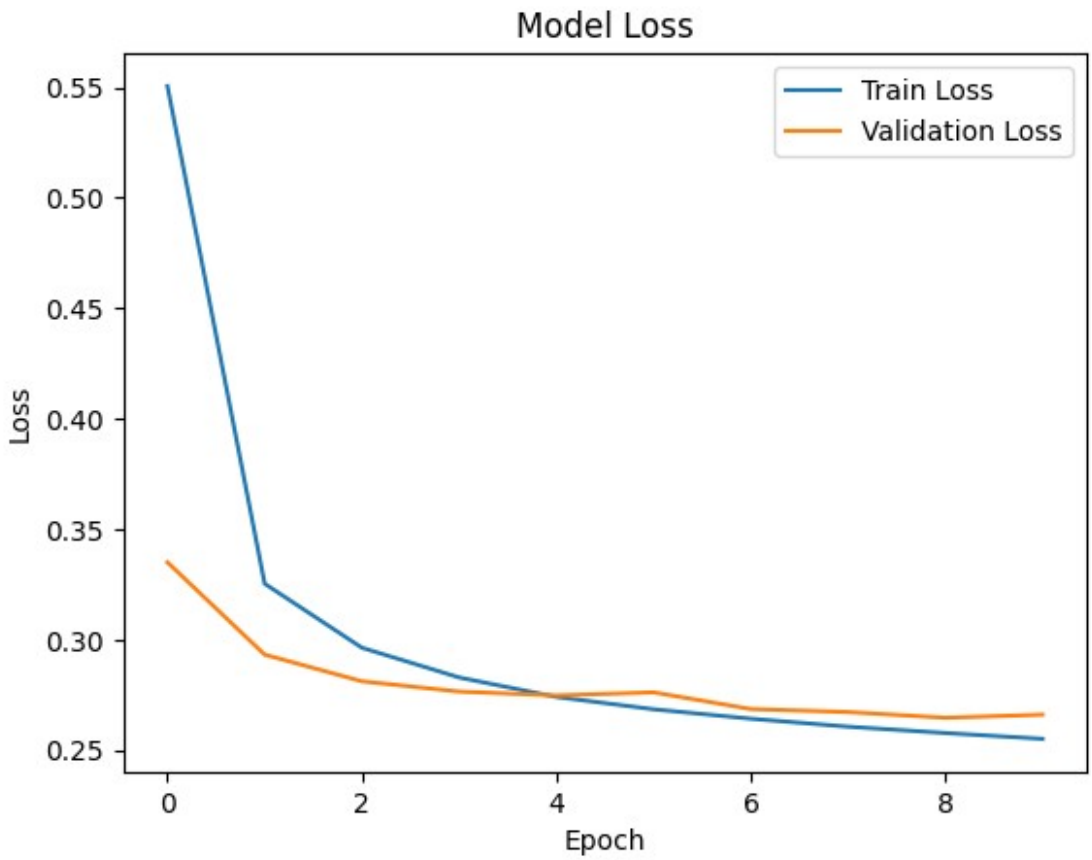
```python
# Model
model = Sequential([
    Dense(10, activation='softmax', input_shape=(784,))
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Train model and store history
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
Model: "sequential_33"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
```

```
┃━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┃━━━━━━━━━━━━━━━━━━━━━━━━━┃━━━━━━━━━━
━━━━━┩
│ dense_33 (Dense)                 │ (None, 10)              │
7,850 │
└──────────────────────────────────┴─────────────────────────┴──────────
─────┘

 Total params: 7,850 (30.66 KB)

 Trainable params: 7,850 (30.66 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.7769 - loss:
0.8626 - val_accuracy: 0.9109 - val_loss: 0.3350
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9087 - loss:
0.3328 - val_accuracy: 0.9194 - val_loss: 0.2932
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9157 - loss:
0.2998 - val_accuracy: 0.9232 - val_loss: 0.2811
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9198 - loss:
0.2865 - val_accuracy: 0.9241 - val_loss: 0.2764
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9237 - loss:
0.2713 - val_accuracy: 0.9234 - val_loss: 0.2747
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9252 - loss:
0.2652 - val_accuracy: 0.9229 - val_loss: 0.2761
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9280 - loss:
0.2564 - val_accuracy: 0.9253 - val_loss: 0.2685
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9297 - loss:
0.2580 - val_accuracy: 0.9259 - val_loss: 0.2672
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9287 - loss:
0.2583 - val_accuracy: 0.9275 - val_loss: 0.2646
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9274 - loss:
0.2626 - val_accuracy: 0.9278 - val_loss: 0.2660
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9164 - loss:
0.2998
Test accuracy: 0.9277999997138977
```


Model Accuracy
0.93
0.92
0.91
0.90
0.89
0.88
0.87
0.86
Accuracy
0
2
4
6
8
Epoch
Train Accuracy
Validation Accuracy



Add next layer using reloc

hidden layer(32,ReLu)

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
```

```
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 5. Build model (Added hidden layer with 32 neurons)
model = Sequential([
    Dense(32, activation='relu', input_shape=(784,)),   # Hidden layer
    Dense(10, activation='softmax')                     # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

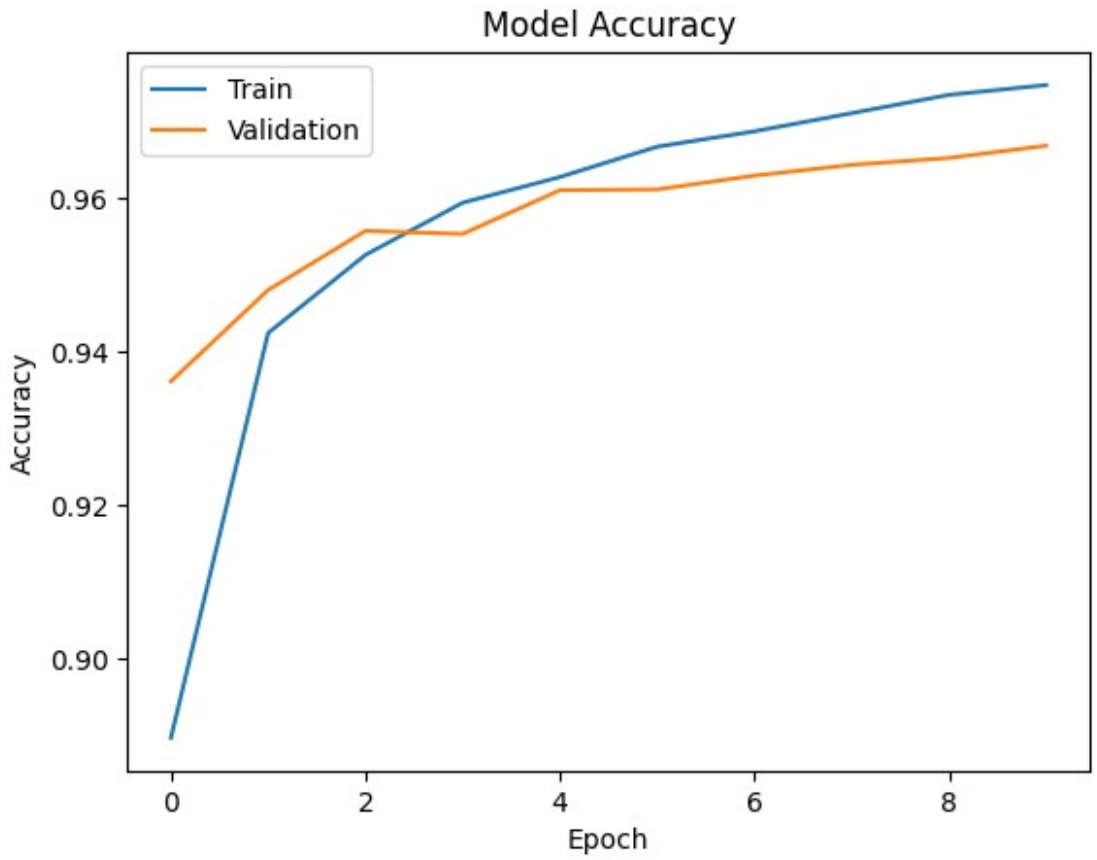
```
Model: "sequential_35"
```

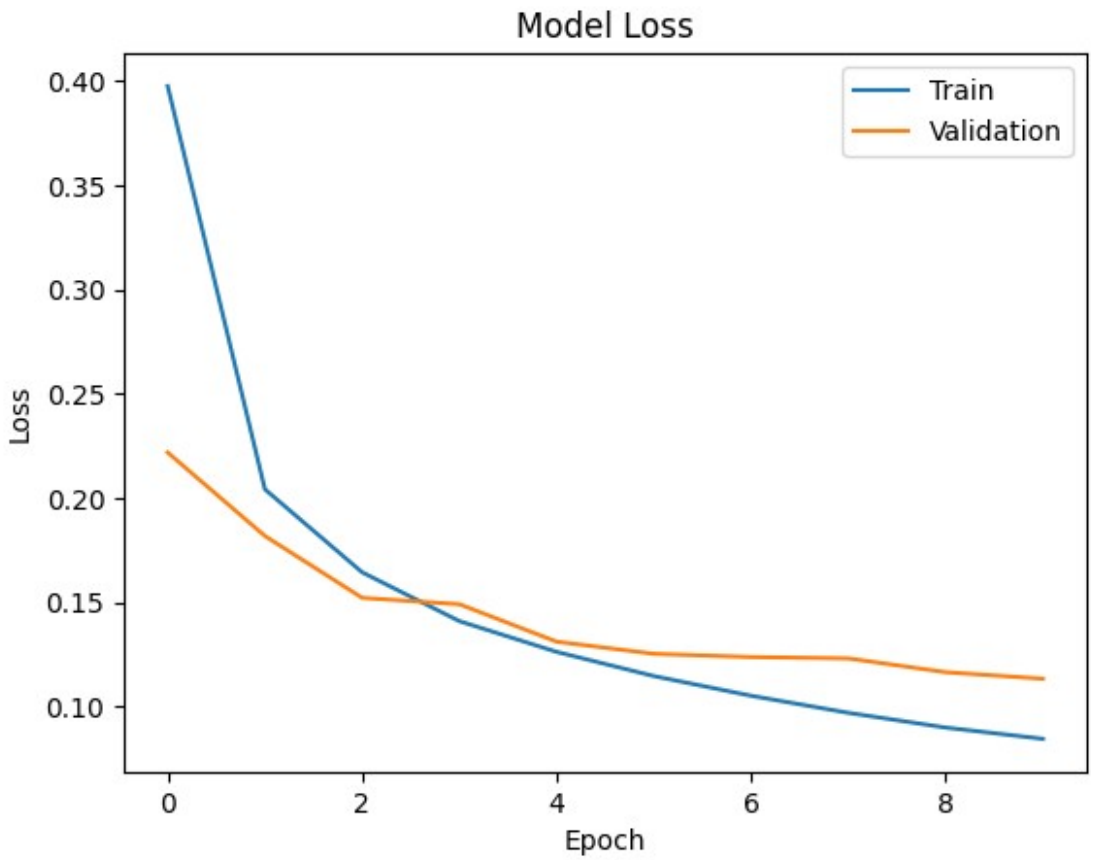
```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ dense_35 (Dense)                │ (None, 32)             │        25,120 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_36 (Dense)                │ (None, 10)             │           330 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 25,450 (99.41 KB)

 Trainable params: 25,450 (99.41 KB)

 Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - accuracy: 0.8079 - loss:
0.6729 - val_accuracy: 0.9361 - val_loss: 0.2218
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9414 - loss:
0.2103 - val_accuracy: 0.9480 - val_loss: 0.1817
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9525 - loss:
0.1650 - val_accuracy: 0.9557 - val_loss: 0.1520
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9592 - loss:
0.1397 - val_accuracy: 0.9553 - val_loss: 0.1490
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9625 - loss:
0.1250 - val_accuracy: 0.9610 - val_loss: 0.1310
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9683 - loss:
0.1077 - val_accuracy: 0.9611 - val_loss: 0.1252
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9695 - loss:
0.1033 - val_accuracy: 0.9629 - val_loss: 0.1236
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9712 - loss:
```

```
0.0967 - val_accuracy: 0.9643 - val_loss: 0.1230
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9744 - loss:
0.0893 - val_accuracy: 0.9652 - val_loss: 0.1163
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9748 - loss:
0.0840 - val_accuracy: 0.9668 - val_loss: 0.1132
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9629 - loss:
0.1283
Test accuracy: 0.9667999744415283
```





Hidden Layer (64->32)

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 64 and 32 neuron layers)
model = Sequential([
    Dense(64, activation='relu', input_shape=(784,)),  # First hidden
layer
    Dense(32, activation='relu'),                      # Second hidden
layer
    Dense(10, activation='softmax')                    # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

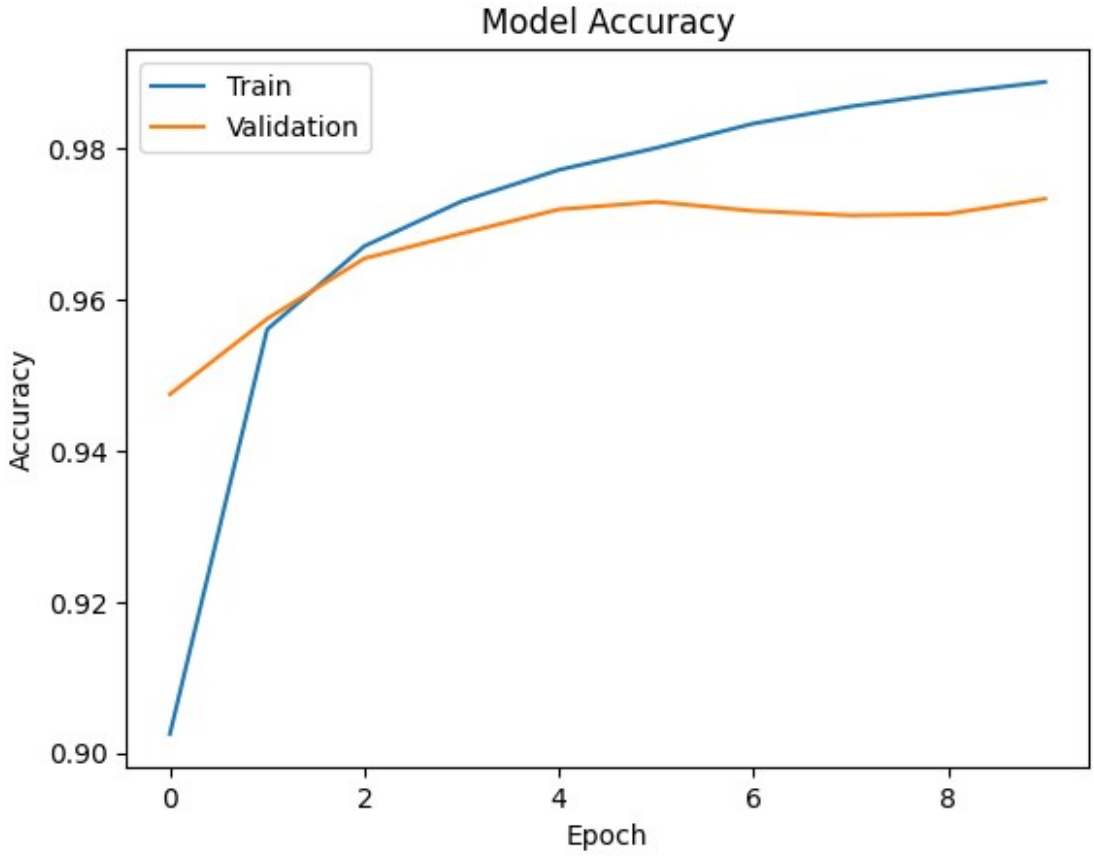
```
Model: "sequential_36"
```

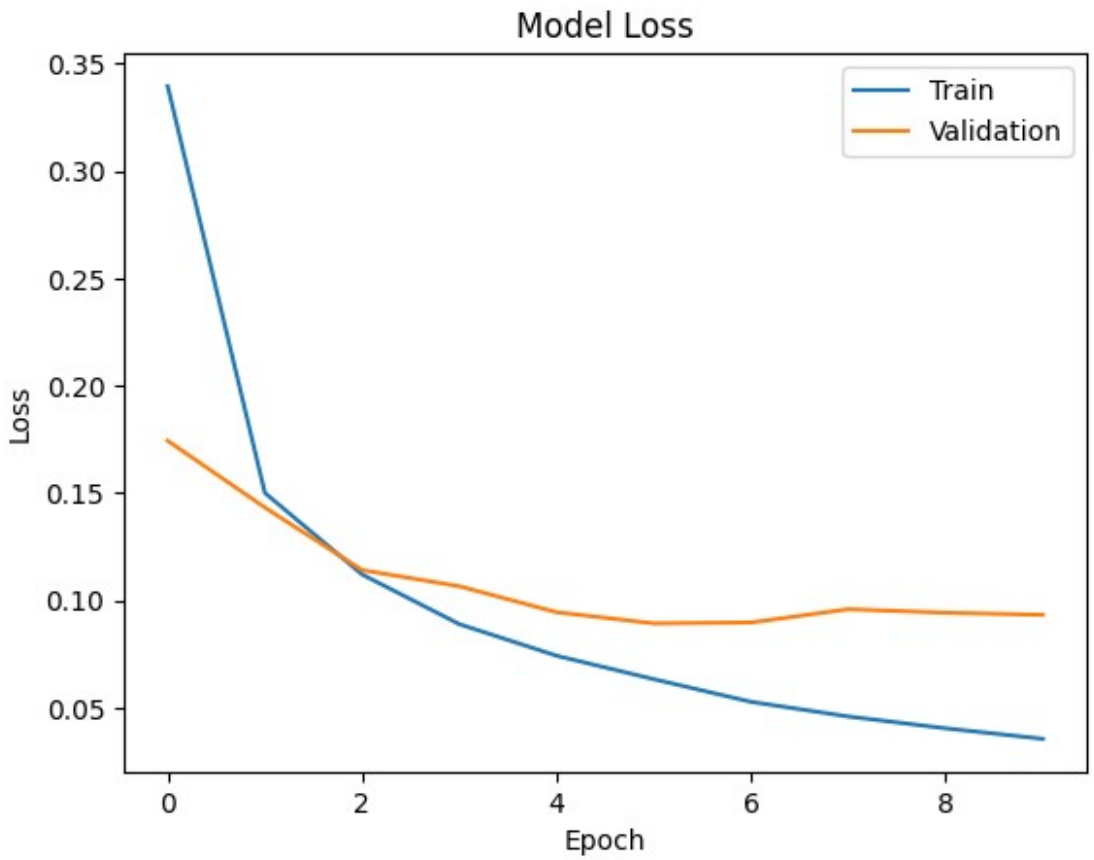
```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ dense_37 (Dense)                     │ (None, 64)                  │          50,240 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_38 (Dense)                     │ (None, 32)                  │           2,080 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_39 (Dense)                     │ (None, 10)                  │             330 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 52,650 (205.66 KB)

 Trainable params: 52,650 (205.66 KB)

 Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.8199 - loss:
0.6128 - val_accuracy: 0.9475 - val_loss: 0.1744
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9528 - loss:
0.1587 - val_accuracy: 0.9575 - val_loss: 0.1433
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9664 - loss:
0.1156 - val_accuracy: 0.9655 - val_loss: 0.1141
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9742 - loss:
0.0859 - val_accuracy: 0.9688 - val_loss: 0.1066
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 7ms/step - accuracy: 0.9780 - loss:
0.0718 - val_accuracy: 0.9720 - val_loss: 0.0945
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - accuracy: 0.9796 - loss:
0.0645 - val_accuracy: 0.9730 - val_loss: 0.0893
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9834 - loss:
```

```
0.0521 - val_accuracy: 0.9718 - val_loss: 0.0897
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9858 - loss:
0.0445 - val_accuracy: 0.9712 - val_loss: 0.0959
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9886 - loss:
0.0381 - val_accuracy: 0.9714 - val_loss: 0.0943
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9898 - loss:
0.0324 - val_accuracy: 0.9734 - val_loss: 0.0933
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9696 - loss:
0.1084
Test accuracy: 0.9733999967575073
```





Hidden layer (128->64->32)

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import matplotlib.pyplot as plt
import numpy as np

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 2. Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0

# 3. Flatten images (28x28 → 784)
X_train = X_train.reshape(-1, 784)
X_test = X_test.reshape(-1, 784)

# 4. One-hot encoding
y_train = to_categorical(y_train, 10)
```

```
y_test = to_categorical(y_test, 10)

# 5. Build model (Added 128, 64, 32 layers)
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),  # Hidden layer
1
    Dense(64, activation='relu'),                       # Hidden layer
2
    Dense(32, activation='relu'),                       # Hidden layer
3
    Dense(10, activation='softmax')                     # Output layer
])

# 6. Compile model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Model summary
model.summary()

# 8. Train model
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_data=(X_test, y_test)
)

# 9. Evaluate model
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test accuracy:", test_acc)

# 10. Plot Accuracy Curve
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Train", "Validation"])
plt.show()

# 11. Plot Loss Curve
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend(["Train", "Validation"])
plt.show()
```

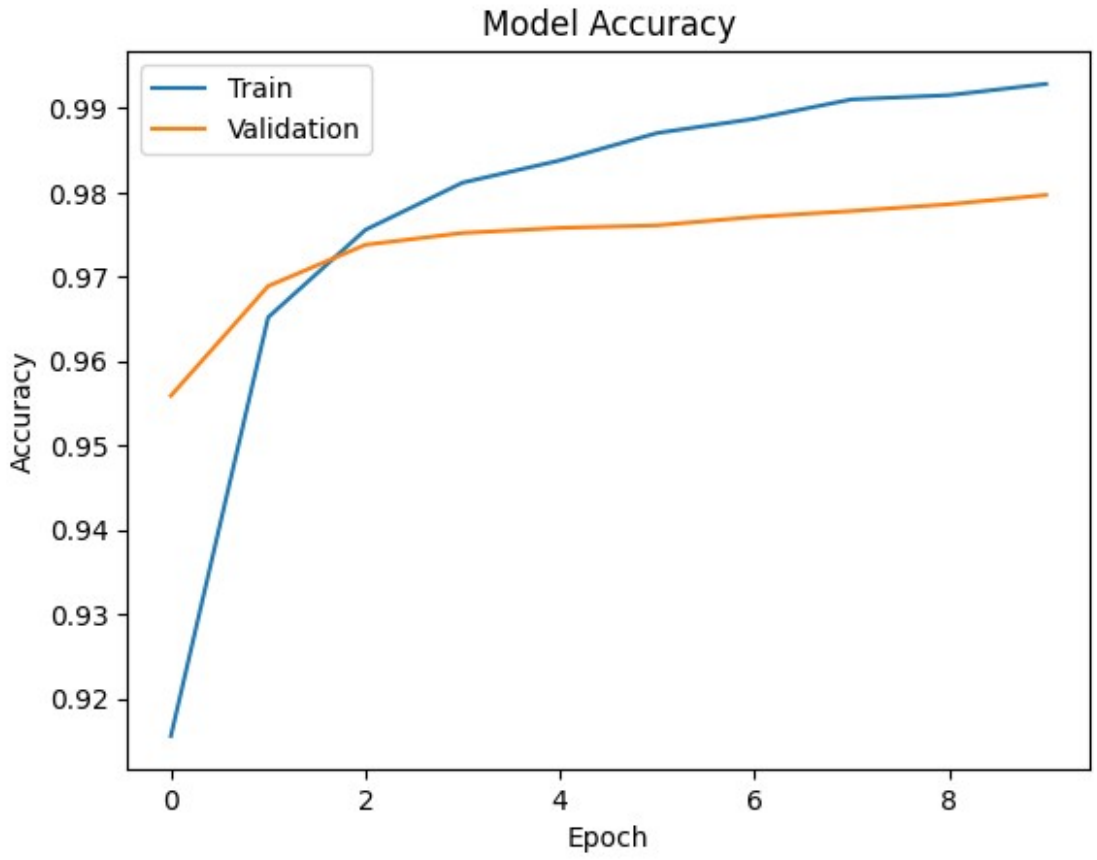
```
Model: "sequential_37"
```

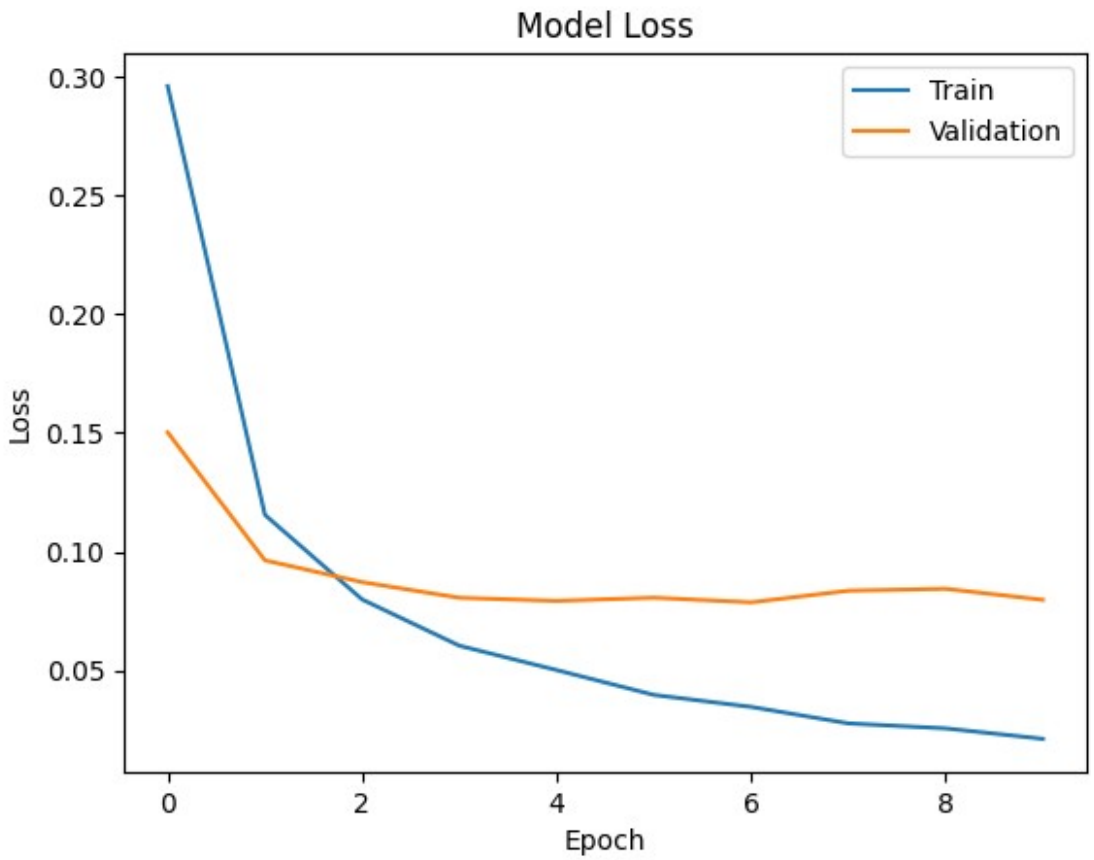
```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ dense_40 (Dense)                │ (None, 128)            │       100,480 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_41 (Dense)                │ (None, 64)             │         8,256 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_42 (Dense)                │ (None, 32)             │         2,080 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_43 (Dense)                │ (None, 10)             │           330 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 111,146 (434.16 KB)

 Trainable params: 111,146 (434.16 KB)

 Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 11s 7ms/step - accuracy: 0.8427 - loss:
0.5522 - val_accuracy: 0.9559 - val_loss: 0.1502
Epoch 2/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9631 - loss:
0.1231 - val_accuracy: 0.9689 - val_loss: 0.0963
Epoch 3/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9753 - loss:
0.0816 - val_accuracy: 0.9738 - val_loss: 0.0871
Epoch 4/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9822 - loss:
0.0567 - val_accuracy: 0.9752 - val_loss: 0.0806
Epoch 5/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9843 - loss:
```

```
0.0466 - val_accuracy: 0.9758 - val_loss: 0.0792
Epoch 6/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9882 - loss:
0.0369 - val_accuracy: 0.9761 - val_loss: 0.0806
Epoch 7/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9893 - loss:
0.0316 - val_accuracy: 0.9771 - val_loss: 0.0786
Epoch 8/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - accuracy: 0.9914 - loss:
0.0263 - val_accuracy: 0.9778 - val_loss: 0.0835
Epoch 9/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.9925 - loss:
0.0220 - val_accuracy: 0.9786 - val_loss: 0.0843
Epoch 10/10
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9933 - loss:
0.0201 - val_accuracy: 0.9797 - val_loss: 0.0797
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9766 - loss:
0.0913
Test accuracy: 0.9797000288963318
```





LAB 9 (19/03/2026)

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([
```

```
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

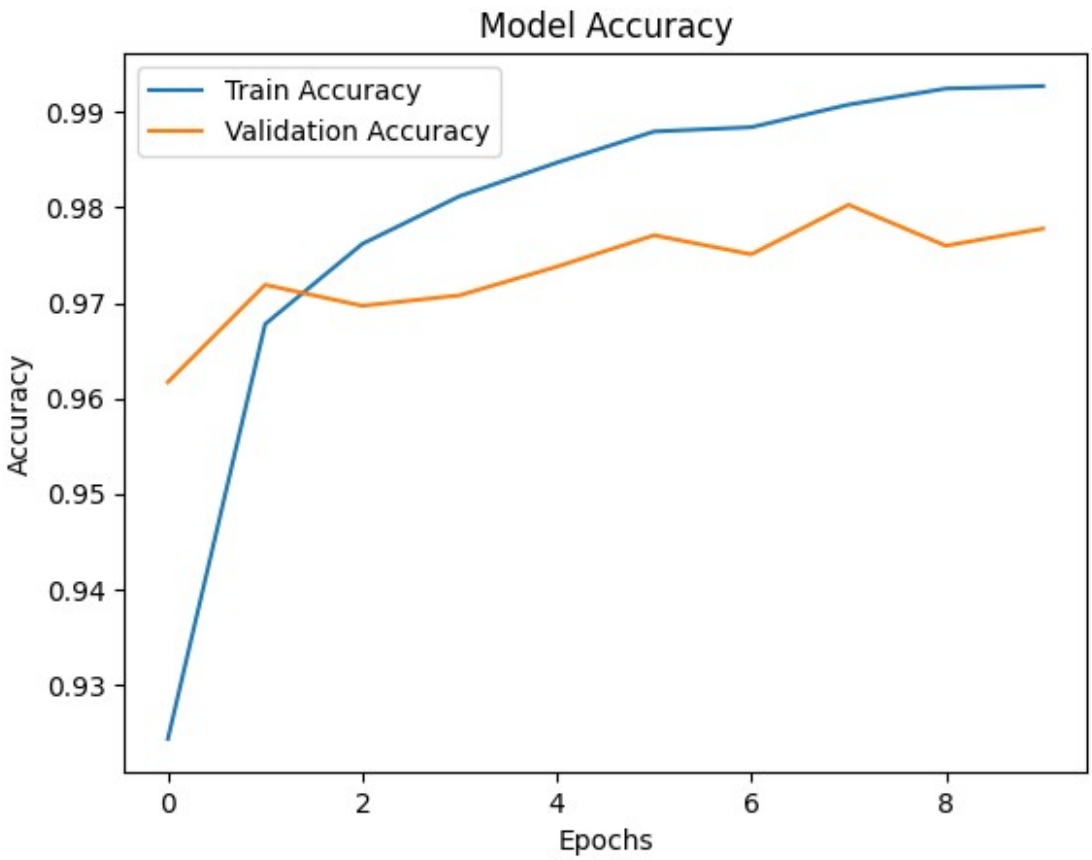
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential_1"
```

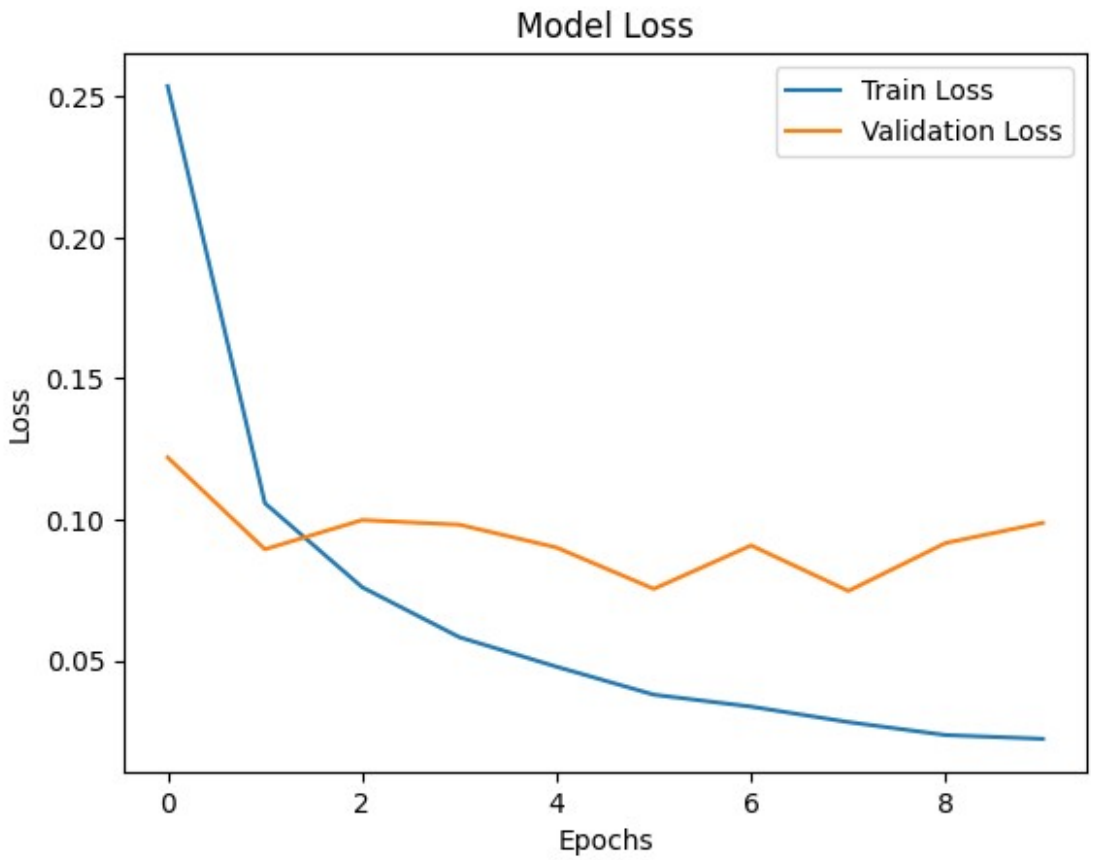
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0

```
├─────────────────────────────────┼────────────────────────┼───────
──────┤
│ conv2d_1 (Conv2D)               │ (None, 11, 11, 64)     │
18,496 │
├─────────────────────────────────┼────────────────────────┼───────
──────┤
│ max_pooling2d_1 (MaxPooling2D)  │ (None, 5, 5, 64)       │
0 │
├─────────────────────────────────┼────────────────────────┼───────
──────┤
│ flatten (Flatten)               │ (None, 1600)           │
0 │
├─────────────────────────────────┼────────────────────────┼───────
──────┤
│ dense_4 (Dense)                 │ (None, 128)            │
204,928 │
├─────────────────────────────────┼────────────────────────┼───────
──────┤
│ dense_5 (Dense)                 │ (None, 10)             │
1,290 │
└─────────────────────────────────┴────────────────────────┴───────
──────┘

 Total params: 225,034 (879.04 KB)

 Trainable params: 225,034 (879.04 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 61s 68ms/step - accuracy: 0.9478 - loss:
0.1724 - val_accuracy: 0.9830 - val_loss: 0.0570
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 62ms/step - accuracy: 0.9852 - loss:
0.0491 - val_accuracy: 0.9890 - val_loss: 0.0382
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 61ms/step - accuracy: 0.9896 - loss:
0.0337 - val_accuracy: 0.9875 - val_loss: 0.0373
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 59s 70ms/step - accuracy: 0.9927 - loss:
0.0238 - val_accuracy: 0.9903 - val_loss: 0.0388
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 75s 62ms/step - accuracy: 0.9939 - loss:
0.0187 - val_accuracy: 0.9917 - val_loss: 0.0363
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9951 - loss:
0.0153 - val_accuracy: 0.9917 - val_loss: 0.0327
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9963 - loss:
0.0113 - val_accuracy: 0.9920 - val_loss: 0.0318
```

```
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 79s 61ms/step - accuracy: 0.9969 - loss:
0.0091 - val_accuracy: 0.9922 - val_loss: 0.0366
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9969 - loss:
0.0080 - val_accuracy: 0.9943 - val_loss: 0.0323
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9981 - loss:
0.0060 - val_accuracy: 0.9935 - val_loss: 0.0346
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9905 - loss:
0.0364
Test accuracy: 0.9904999732971191
```

```python
import matplotlib.pyplot as plt

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(['Train Loss', 'Validation Loss'])
plt.show()
```


Model Accuracy
Train Accuracy
Validation Accuracy
Accuracy
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0
2
4
6
8
Epochs




```
# 1. Import Libraries
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
import matplotlib.pyplot as plt
import numpy as np

# 2. Load Dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# 3. Normalization
X_train = X_train / 255.0
X_test = X_test / 255.0

# 4. Train/Test already split in MNIST

# 5. Reshape for CNN (important)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

# 6. Build Model (Conv → Pool → Flatten → Dense)
```

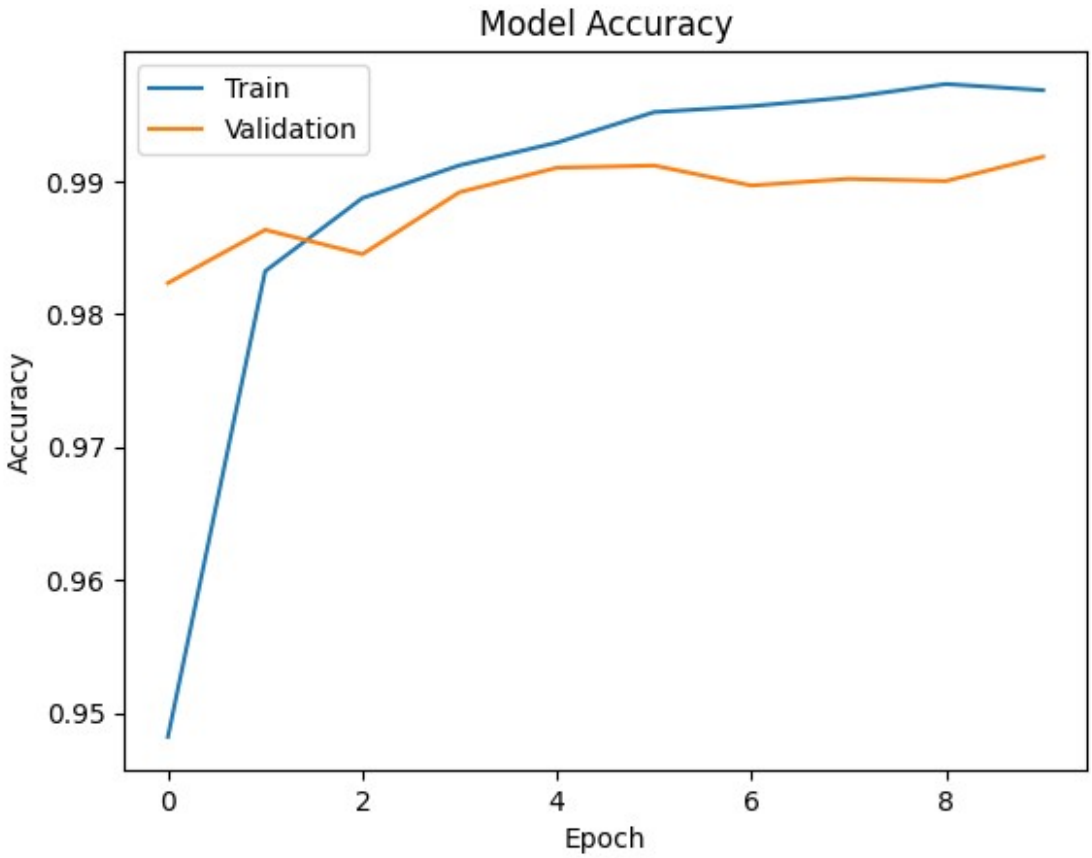
```
model = Sequential()

# Convolution
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))

# Pooling
model.add(MaxPooling2D((2,2)))

# Convolution
model.add(Conv2D(64, (3,3), activation='relu'))

# Pooling
model.add(MaxPooling2D((2,2)))

# Flatten
model.add(Flatten())

# Dense
model.add(Dense(128, activation='relu'))

# Output Dense
model.add(Dense(10, activation='softmax'))

# 7. Compile
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 8. Summary
model.summary()

# 9. Training
history = model.fit(X_train, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)

# 10. Evaluation
test_loss, test_acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_acc)

# 11. Graphs (Loss & Accuracy)

# Accuracy Graph
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
```

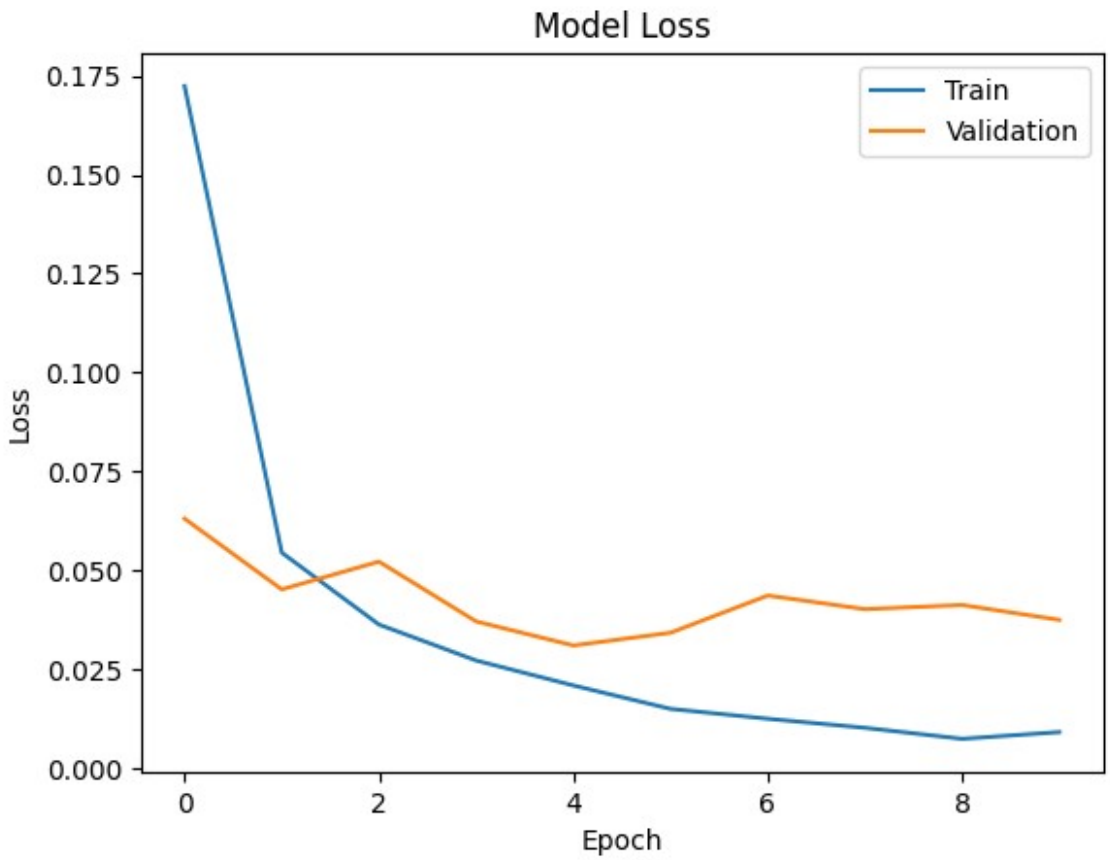
```
plt.legend(['Train', 'Validation'])
plt.show()

# Loss Graph
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```

```
Model: "sequential_2"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃       Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━┩
│ conv2d_2 (Conv2D)               │ (None, 26, 26, 32)     │           320 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d_2 (MaxPooling2D)  │ (None, 13, 13, 32)     │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ conv2d_3 (Conv2D)               │ (None, 11, 11, 64)     │        18,496 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ max_pooling2d_3 (MaxPooling2D)  │ (None, 5, 5, 64)       │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ flatten_1 (Flatten)             │ (None, 1600)           │             0 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_6 (Dense)                 │ (None, 128)            │       204,928 │
├─────────────────────────────────┼────────────────────────┼───────────────┤
│ dense_7 (Dense)                 │ (None, 10)             │         1,290 │
└─────────────────────────────────┴────────────────────────┴───────────────┘
```

```
 Total params: 225,034 (879.04 KB)

 Trainable params: 225,034 (879.04 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 63s 71ms/step - accuracy: 0.9482 - loss:
0.1724 - val_accuracy: 0.9823 - val_loss: 0.0631
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 51s 60ms/step - accuracy: 0.9832 - loss:
0.0546 - val_accuracy: 0.9863 - val_loss: 0.0453
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 62ms/step - accuracy: 0.9887 - loss:
0.0364 - val_accuracy: 0.9845 - val_loss: 0.0523
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 64ms/step - accuracy: 0.9912 - loss:
0.0273 - val_accuracy: 0.9892 - val_loss: 0.0372
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 81s 62ms/step - accuracy: 0.9929 - loss:
0.0210 - val_accuracy: 0.9910 - val_loss: 0.0311
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 64ms/step - accuracy: 0.9952 - loss:
0.0151 - val_accuracy: 0.9912 - val_loss: 0.0343
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 64ms/step - accuracy: 0.9956 - loss:
0.0126 - val_accuracy: 0.9897 - val_loss: 0.0437
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 52s 62ms/step - accuracy: 0.9963 - loss:
0.0103 - val_accuracy: 0.9902 - val_loss: 0.0403
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 53s 63ms/step - accuracy: 0.9973 - loss:
0.0075 - val_accuracy: 0.9900 - val_loss: 0.0413
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 63ms/step - accuracy: 0.9968 - loss:
0.0092 - val_accuracy: 0.9918 - val_loss: 0.0376
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step - accuracy: 0.9895 - loss:
0.0393
Test Accuracy: 0.9894999861717224
```


Model Accuracy
Train
Validation
0.99
0.98
0.97
0.96
0.95
Accuracy
0
2
4
6
8
Epoch




```
import matplotlib.pyplot as plt

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
```

```
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Acc
(batch={batch})')
    plt.plot(histories[batch].history['val_accuracy'], linestyle='--',
label=f'Val Acc (batch={batch})')

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()


Training with batch size: 32
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 71s 39ms/step - accuracy: 0.9585 -
loss: 0.1375 - val_accuracy: 0.9870 - val_loss: 0.0475
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 62s 37ms/step - accuracy: 0.9855 -
loss: 0.0449 - val_accuracy: 0.9883 - val_loss: 0.0444
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 38ms/step - accuracy: 0.9904 -
loss: 0.0310 - val_accuracy: 0.9912 - val_loss: 0.0348
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 37ms/step - accuracy: 0.9925 -
loss: 0.0222 - val_accuracy: 0.9898 - val_loss: 0.0413
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 60s 35ms/step - accuracy: 0.9946 -
loss: 0.0158 - val_accuracy: 0.9900 - val_loss: 0.0340
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 63s 37ms/step - accuracy: 0.9953 -
loss: 0.0133 - val_accuracy: 0.9913 - val_loss: 0.0399
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9965 -
loss: 0.0099 - val_accuracy: 0.9910 - val_loss: 0.0359
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 66s 39ms/step - accuracy: 0.9970 -
loss: 0.0088 - val_accuracy: 0.9908 - val_loss: 0.0441
```

```
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 62s 37ms/step - accuracy: 0.9975 -
loss: 0.0073 - val_accuracy: 0.9922 - val_loss: 0.0364
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 61s 36ms/step - accuracy: 0.9976 -
loss: 0.0072 - val_accuracy: 0.9917 - val_loss: 0.0425

Training with batch size: 64
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 61s 70ms/step - accuracy: 0.9468 - loss:
0.1734 - val_accuracy: 0.9847 - val_loss: 0.0523
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9839 - loss:
0.0512 - val_accuracy: 0.9857 - val_loss: 0.0476
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 54s 64ms/step - accuracy: 0.9888 - loss:
0.0354 - val_accuracy: 0.9903 - val_loss: 0.0376
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9917 - loss:
0.0261 - val_accuracy: 0.9873 - val_loss: 0.0393
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9932 - loss:
0.0208 - val_accuracy: 0.9907 - val_loss: 0.0344
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 83s 66ms/step - accuracy: 0.9949 - loss:
0.0162 - val_accuracy: 0.9900 - val_loss: 0.0392
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 56s 66ms/step - accuracy: 0.9956 - loss:
0.0127 - val_accuracy: 0.9927 - val_loss: 0.0309
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 56s 66ms/step - accuracy: 0.9968 - loss:
0.0100 - val_accuracy: 0.9912 - val_loss: 0.0376
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 55s 65ms/step - accuracy: 0.9972 - loss:
0.0086 - val_accuracy: 0.9903 - val_loss: 0.0366
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 82s 65ms/step - accuracy: 0.9969 - loss:
0.0088 - val_accuracy: 0.9915 - val_loss: 0.0411

Training with batch size: 128
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 51s 117ms/step - accuracy: 0.9356 - loss:
0.2175 - val_accuracy: 0.9808 - val_loss: 0.0679
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 48s 113ms/step - accuracy: 0.9806 - loss:
0.0619 - val_accuracy: 0.9852 - val_loss: 0.0519
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 116ms/step - accuracy: 0.9870 - loss:
0.0428 - val_accuracy: 0.9882 - val_loss: 0.0424
```

```
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 113ms/step - accuracy: 0.9902 - loss:
0.0312 - val_accuracy: 0.9882 - val_loss: 0.0392
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 112ms/step - accuracy: 0.9918 - loss:
0.0255 - val_accuracy: 0.9900 - val_loss: 0.0376
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 116ms/step - accuracy: 0.9934 - loss:
0.0200 - val_accuracy: 0.9873 - val_loss: 0.0447
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 112ms/step - accuracy: 0.9947 - loss:
0.0159 - val_accuracy: 0.9893 - val_loss: 0.0408
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 48s 115ms/step - accuracy: 0.9957 - loss:
0.0138 - val_accuracy: 0.9908 - val_loss: 0.0349
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 83s 116ms/step - accuracy: 0.9970 - loss:
0.0096 - val_accuracy: 0.9892 - val_loss: 0.0432
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 81s 114ms/step - accuracy: 0.9972 - loss:
0.0082 - val_accuracy: 0.9910 - val_loss: 0.0405
```